

Chapter 4. Structured Concurrency

Simplicity is prerequisite for reliability.

—Edsger W. Dijkstra

In our previous chapters, we have seen how virtual threads open up a world of possibilities by letting us handle blocking tasks in Java concurrency. With thousands of lightweight virtual threads capable of spawning without significant overhead, blocking calls are no longer a concern. However, one critical issue still needs to be addressed: managing the relationships and dependencies between tasks when they are broken down and distributed across multiple threads.

Imagine a scenario where a parent task has several dependent subtasks, each running on a different thread. If the parent task terminates unexpectedly, its subtasks continue to execute independently. The results of these subtasks become irrelevant, yet they still consume resources.

Moreover, if the parent must be completed only when all its subtasks are complete, any failure among the subtasks should ideally lead to the cancellation of the entire operation. This lack of coordination can lead to numerous orphaned threads. These orphaned threads will exhaust valuable memory and processor time by doing unnecessary work. Often, such inefficiencies involve considerable costs.

In traditional concurrency models, such as Java's `ExecutorService` and `Future`, task hierarchies are not inherently supported, which makes it difficult to propagate cancellations or handle errors in a structured manner. This results in a fragmented and often error-prone approach to concurrent programming.

Structured concurrency is a paradigm that addresses these challenges by treating groups of related tasks as a single unit of work. It ensures that the lifecycles of subtasks are bound to the parent task, providing a transparent and manageable structure for concurrent execution. This not only simplifies error handling and cancellation but also enhances the reliability and observability of concurrent code.

In this chapter, we will explore structured concurrency in Java, examining its benefits and how it addresses the limitations of traditional concurrency models. We will introduce the `StructuredTaskScope` API, discuss

how it helps manage task hierarchies, and provide practical examples to illustrate its use. By the end of this chapter, you will have a solid understanding of how to implement structured concurrency in your Java applications and how to write more robust and maintainable concurrent code.

The Challenge of Unstructured Concurrency

In Java, concurrency has traditionally been managed using abstractions like `ExecutorService` and `Future`. While these classes provide a means of executing tasks concurrently, they also present some challenges, especially when tasks must be coordinated.

Consider a common scenario: a web application needs to fetch product details and associated reviews for display. An approach using `ExecutorService` could be implemented to perform these operations in parallel, thereby improving response time.

To illustrate this, let's construct the Java code for our scenario piece by piece. We'll begin by defining the data structures that will hold our product information and reviews. Java records are well-suited for these simple data carriers due to their conciseness and simplicity. We'll also define a custom `ProductServiceException` to more effectively handle errors specific to our product service logic:

```
record Product(Long id, String name, String description) {}
record Review(Long id, String comment, int rating, Long productId) {}
record ProductInfo(Product product, List<Review> reviews) {}

class ProductServiceException extends RuntimeException {
    public ProductServiceException(String message, Throwable cause) {
        super(message, cause);
    }
    public ProductServiceException(String message) {
        super(message);
    }
}
```

With our data models and custom exception in place, we need to simulate the actual work of fetching data. Real-world data retrieval typically involves network latency, which can be mimicked using helper methods. The `ProductService` class, which we'll define shortly, will include these private helper methods: `sleepForAWhile()` to introduce a delay, `log()` to log messages with a concise thread identifier for better readability, and `fetchProduct()` and `fetchReviews()` to simulate the

data fetching operations. These methods will print messages indicating their progress, including the name of the executing thread, to help visualize concurrency.

Now, we can implement the central `ProductService` class. The key method here is `fetchProductInfo()`. This method utilizes an `ExecutorService` (specifically, `Executors.newVirtualThreadPerTaskExecutor()`, which is suitable for I/O-bound operations as it creates lightweight virtual threads for each task) to concurrently run the product- and review-fetching tasks. It then uses `Future.get()` to await their completion and retrieve their results. The helper methods for simulating work are included as private instance methods within this class for proper encapsulation:

```
public class ProductService {
    public ProductInfo fetchProductInfo(Long productId) {
        log("Fetching product & reviews for id: " + productId);

        try (var ExecutorService
            = Executors.newVirtualThreadPerTaskExecutor()) { ❶

            Future<Product> productTask =
                ExecutorService.submit(() -> fetchProduct(productId)); ❷
            Future<List<Review>> reviewsTask =
                ExecutorService.submit(() -> fetchReviews(productId)); ❸
            Product product = productTask.get(); ❹
            log("Product retrieved for id: " + productId);
            List<Review> reviews = reviewsTask.get(); ❺
            log("Reviews retrieved for id: " + productId);
            log("All info fetched for id: " + productId);
            return new ProductInfo(product, reviews); ❻

        } catch (ExecutionException | InterruptedException e) { ❼
            Throwable cause = e.getCause() != null ? e.getCause() : e;
            log("Error processing product info for id: " +
                productId + ": " + cause.getMessage());

            if (e instanceof InterruptedException) {
                Thread.currentThread().interrupt();
            }

            throw new ProductServiceException(
                "Fetch failed for id: " + productId, cause);
        }
    }
}
```

Now let's go through what we have done here:

We created a virtual thread executor that spawns a new virtual thread for each submitted task.

- ❷ Submits the product-fetching operation to run concurrently.
- ❸ Submits the review-fetching operation to run concurrently.
- ❹ Blocks until the product task completes and retrieves the result.
- ❺ Blocks until the review task completes and retrieves the result.
- ❻ Combines both results into the final `ProductInfo` object.
- ❼ Catches both execution and interruption exceptions with appropriate handling.

To complete our concurrent product service example, we need several supporting methods that simulate real-world operations:

```
private Product fetchProduct(Long productId) {
    log("Fetching product id: " + productId);
    sleepForAWhile(Duration.ofSeconds(1)); // Simulate network call
    return new Product(productId, "Sample Product",
        "A great product description.");
}

private List<Review> fetchReviews(Long productId) {
    log("Fetching reviews for id: " + productId);
    sleepForAWhile(Duration.ofSeconds(2)); // Simulate network call
    return List.of(
        new Review(1L, "Excellent!", 5, productId),
        new Review(2L, "Good value.", 4, productId)
    );
}

private void sleepForAWhile(Duration duration) {
    try {
        Thread.sleep(duration);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new RuntimeException("Thread interrupted during sleep", e);
    }
}

private static void log(String message) {
    Thread currentThread = Thread.currentThread();
    String threadName = currentThread.isVirtual()
        ? "VThread[# " + currentThread.threadId() + "]"
        : currentThread.getName(); ❶
    String currentTime = LocalTime.now()
        .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
    System.out.printf("%s %-15s: %s%n", currentTime, threadName, message);
}

void main() { ❷
    ProductService productService = new ProductService();
    long testProductId = 1L;
```



```

        log("Attempting to fetch product info for ID: " + testProductId);
    }
    try {
        ProductInfo productInfo = productService.fetchProductInfo(testProductId);
        log("Successfully retrieved: " + productInfo);
    } catch (ProductServiceException e) {
        log("Service Error: " + e.getMessage() +
            (e.getCause() != null ? " | Caused by: " +
            e.getCause().getMessage() : ""));
    }
}
}

```

Let's examine the key implementation details in this code:

- ❶ This custom logging format helps us track which threads are executing our concurrent operations. Virtual threads generally don't have a name, but their `toString()` method gives us verbose names like `VirtualThread[#34]/runnable@ForkJoinPool-1-worker-1`, and we shorten this to `VThread[#34]` for clarity while still distinguishing virtual threads from platform threads.
- ❷ Runs the complete example, showing both the concurrent execution flow and error handling, producing the output we analyze next.

With all components in place, let's execute the complete example to observe the concurrent behavior. Run the following command in your terminal to see how virtual threads handle our parallel operations:

```
java ProductService.java
```

And the output would look as follows:

```

00:50:27.625 main          : Attempting to fetch product info for ID: 1
00:50:27.627 main          : Fetching product & reviews for id: 1
00:50:27.631 VThread[#34] : Fetching product id: 1
00:50:27.631 VThread[#36] : Fetching reviews for id: 1
00:50:28.638 main          : Product retrieved for id: 1
00:50:29.652 VThread[#36] : Fetched reviews for id: 1
00:50:29.653 main          : Reviews retrieved for id: 1
00:50:29.653 main          : All info fetched for id: 1
00:50:29.660 main          : Successfully retrieved Product Info:
ProductInfo[product=Product[id=1, name=Sample Product, description=A great
product description.], reviews=[Review[id=1, comment=Excellent!, rating=5,
productId=1], Review[id=2, comment=Good value., rating=4, productId=1]]]

```

At first glance, this approach seems perfectly reasonable. We've split the work into two concurrent tasks: fetching the product details and fetching its reviews. However, if we carefully consider this, we have a few prob-

lems here. As both methods execute concurrently, each method can succeed or fail independently.

Imagine a scenario where the product database is temporarily unavailable. The `fetchProduct()` task might throw an exception, which will end up stopping the main request thread. However, oblivious to this failure, the `fetchReviews()` task continues to execute in its own thread. This is a thread leak that can potentially hog resources and add unnecessary load to the server ([Figure 4-1](#)).

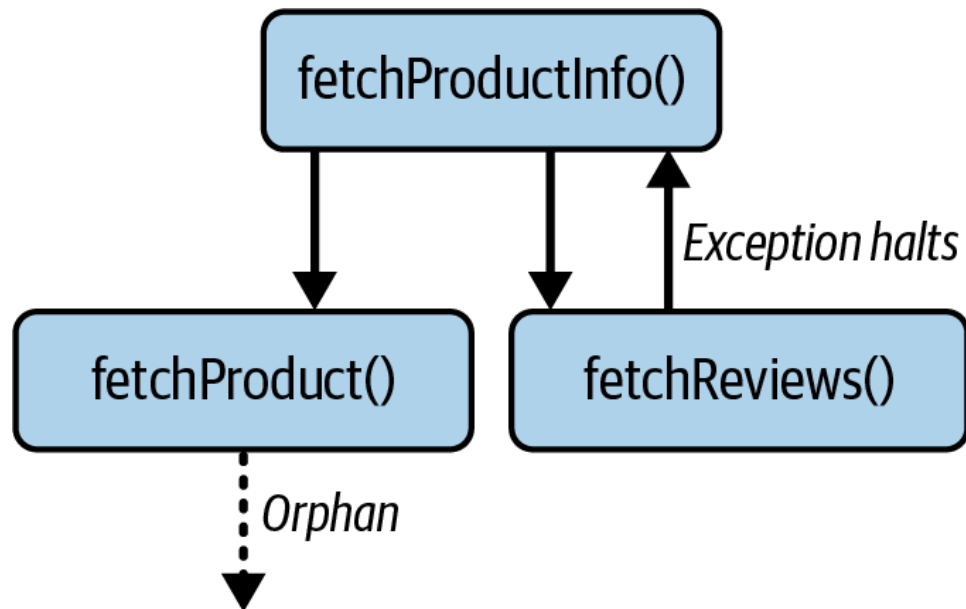


Figure 4-1. Parent task spawns two child tasks; however, an exception in one child halts the parent, leaving the other child task orphaned

If the user decides to abandon the request, the server might want to halt the request thread by interrupting. However, this won't cancel the sub-tasks; the `fetchProduct()` and `fetchReviews()` tasks will continue running even after the `fetchProductInfo()` method fails. This results in two threads leaking and a waste of precious CPU cycles on a request that's no longer relevant.

The `fetchProductInfo()` method returns results based on the results of both subsequent methods. So, the `fetchProductInfo()` method either succeeds with both results or fails. However, what if `fetchProduct()` takes a significantly longer time than `fetchReviews()`, and while it's fetching the product, `fetchReviews()` fails? In this case, the `fetchProductInfo()` method unnecessarily waits for `fetchProduct()`, which will ultimately be abandoned.

To illustrate these problems in action, let's modify our methods to simulate a realistic failure scenario. We'll create a situation where one task takes significantly longer while the other fails quickly:

```
private Product fetchProduct(Long productId) {  
    log("Fetching product id: " + productId);
```

```

        if (productId == 1L) {
            log("Product id: " + productId
                + " - simulating long network call (5 seconds).");
            sleepForAWhile(Duration.ofSeconds(5));
            log("Product id: " + productId + " fetch complete.");
            return new Product(productId,
                "Long-Fetched Product", "This product takes time to fetch.");
        }
        log("Product id: " + productId +
            " - simulating standard network call (1 second).");
        sleepForAWhile(Duration.ofSeconds(1));
        log("Product id: " + productId + " fetch complete.");
        return new Product(productId, "Sample Product",
            "A great product description.");
    }

    private List<Review> fetchReviews(Long productId) {
        log("Fetching reviews for id: " + productId);
        if (productId == 1L) {
            log("Reviews for id: " + productId
                + " - simulating quick failure after 1 second.");
            sleepForAWhile(Duration.ofSeconds(1));
            throw new ProductServiceException("Simulated failure " +
                "fetching reviews for product " + productId);
        }
        log("Reviews for id: " + productId
            + " - simulating network call (2 seconds).");
        sleepForAWhile(Duration.ofSeconds(2));
        List<Review> reviews = List.of(
            new Review(1L, "Excellent!", 5, productId),
            new Review(2L, "Good value.", 4, productId)
        );
        log("Fetched reviews for id: " + productId);
        return reviews;
    }
}

```

In the preceding method, we've introduced a conditional behavior based on `productId == 1L` to create a controlled scenario: the product fetch will take five seconds (simulating a slow database), while the reviews fetch will fail after just one second. This timing mismatch perfectly demonstrates our resource leak problem.

Now, if we rerun the code, the output would be as follows:

```

00:58:02.885 main           : Attempting to fetch product info for ID: 1
00:58:02.887 main           : Fetching product & reviews for id: 1
00:58:02.890 VThread[#36]   : Fetching reviews for id: 1
00:58:02.890 VThread[#34]   : Fetching product id: 1
00:58:02.890 VThread[#36]   : Reviews for id: 1 - simulating quick failure af
1 second.
00:58:02.891 VThread[#34]   : Product id: 1 - simulating long network call
(5 seconds).

```

```
00:58:07.899 VThread[#34] : Product id: 1 fetch complete.
00:58:07.900 main       : Product retrieved for id: 1
00:58:07.905 main       : Error processing product info for id:
1: Simulated failure fetching reviews for product 1
00:58:07.906 main       : Service Error: Fetch failed for id:
1 | Caused by: Simulated failure fetching reviews for product 1
```

The log output reveals the exact problem we described. Even though `fetchReviews()` throws an exception, the `productFetch()` method continues its execution, taking a full five seconds to retrieve the product. This delay is unnecessary because the result can no longer be used.

This example exposes the core issue with unstructured concurrency. It treats tasks as independent entities, ignoring their relationships and dependencies. The result is a breeding ground for error, resource leaks, and performance bottlenecks. Even further down the road, it makes it difficult to diagnose issues. Observability tools such as thread dumps will show each method call as stacks of unrelated threads with no hint of the task-subtask relationship.

It's like the `goto` statement back in the old days. As a result of using `goto`, the execution jumps to arbitrary points in the code, making it challenging to trace the program's logic. Similarly, tasks or threads can be executed in an arbitrary order in unstructured concurrency, making it hard to predict the program's state at any given time.

Structured concurrency represents a paradigm shift that can solve these problems, making programming in the presence of concurrency clean and straightforward.

The Promise of Structured Concurrency

Structured concurrency arrives as a fresh approach to Java concurrency. Rather than treating concurrent tasks as independent entities, it organizes them in a way that mirrors the logical structure of our programs.

The core principle is straightforward: when a task spawns multiple concurrent subtasks, they must all return to the same point in the parent task's code. Think of it like a manager delegating work to team members; everyone reports back before the project can be considered complete.

In this model, the parent task acts as a supervisor, monitoring its subtasks and waiting for their completion or intervening when problems arise. This constraint may seem simple, but it has profound implications for the reliability and maintainability of concurrent code. Every subtask, regard-

less of whether it succeeds or fails, must report back to its parent before the operation can conclude.

Now let's discuss some of the benefits that it brings.

First, structured concurrency provides a unified mechanism for error and cancellation handling across a group of related (sibling) tasks. For example, if one subtask encounters an error or is canceled, its sibling tasks are automatically terminated, and the error propagates gracefully up the task hierarchy.

Second, it prevents thread leaks by enforcing a strict parent-child lifecycle for tasks. Tasks cannot outlive their intended scope because the parent ensures all subtasks are properly cleaned up upon completion, leading to more predictable system behavior.

Furthermore, structured concurrency enhances observability by providing a clear and intuitive representation of task dependencies. Tools such as thread dumps and specialized profilers can then easily trace these task relationships. This improved traceability facilitates faster troubleshooting and root cause analysis when issues arise.

The paradigm also encourages a more declarative programming style, allowing developers to focus on the business logic of their tasks rather than the intricate details of thread management. This shift results in cleaner, more maintainable code that is easier to understand and modify, which ultimately reduces development time and costs.

Finally, because structured concurrency often works on top of virtual threads, this combination enables the creation of a vast number of concurrent tasks with minimal overhead. Structured concurrency provides the essential framework for orchestrating these virtual threads, ensuring their safe and efficient execution, even at a large scale.

Structured concurrency is positioned to become a fundamental component of Java concurrent programming. By addressing the core problems of unstructured concurrency, it offers a path toward more reliable and maintainable systems.

Let's examine the Structured Concurrency API and see how it addresses the challenges we've discussed.

Understanding the API

At the heart of the Structured Concurrency API lies the StructuredTaskScope in the [java.util.concurrent](#) package. In this sec-

tion, we'll explore its main features, how it works, and how it can be used to simplify and improve concurrent programming in Java.

StructuredTaskScope

The StructuredTaskScope interface is central to the structured concurrency model in Java. It allows developers to manage groups of related tasks, ensuring that their lifecycles are bound together. This makes error handling, cancellation, and result aggregation more straightforward and reliable.

The `StructuredTaskScope` is a sealed interface, parameterized by the types `T` and `R`, which represent the result type of the tasks executed within the scope:

```
public sealed interface StructuredTaskScope<T,R>
    extends AutoCloseable
```

Let's explore its key methods:

```
<U extends T> StructuredTaskScope.Subtask<U>
fork(Callable<? extends U> task)
```

Creates and starts a new subtask within the scope. The task is executed concurrently in a separate thread (typically a virtual thread). The method returns a Subtask object representing the running subtask.

```
<U extends T> StructuredTaskScope.Subtask<U>
fork(Runnable task)
```

Creates and starts a new subtask within the scope to run the provided action. The action executes concurrently in a separate thread (typically a virtual thread). Because the task is a `Runnable`, it produces no result; therefore, `Subtask.get()` returns `null` on successful completion. The method returns a `Subtask` object representing the running subtask.

```
join()
```

Returns the result after waiting for all subtasks to complete or the scope to be canceled. This method blocks the calling thread (which must be the scope owner) until all subtasks finish. If a timeout is configured and expires, the scope is canceled and

`TimeoutException` is thrown. After waiting completes, the joiner's `result()` method is invoked to get the result or throw an exception. If the `result()` method throws an exception,

`FailedException` is thrown with that exception as the cause.

This method can only be called once by the scope owner.

```
close()
```

This method closes the scope, ensuring that all subtasks have been completed or canceled. It is typically called implicitly when using the `try-with-resources` statement.

The `StructuredTaskScope` is an interface; it cannot be instantiated directly. However, a key feature of this API is the static factory method `open()`. This method has the following signature:

```
static <T> StructuredTaskScope<T,Void> open()
```

This method creates a new `StructuredTaskScope` that can be used to fork subtasks returning results of any type. The scope's `join()` method waits for all subtasks to complete successfully or for any subtask to fail. While `open()` has other overloaded methods, we will discuss those in a subsequent section. Because `StructuredTaskScope` extends `AutoCloseable`, we will use it within a `try-with-resources` block, ensuring automatic closure. It's important to note that the `close()` method should not be invoked manually in such cases.

TIP

`StructuredTaskScope` is intended to be used in a strictly structured manner. If its `close()` method is invoked before all nested task scopes (if we have them) have been closed, the scope will: 1. Attempt to close the underlying construct of each nested scope (in the reverse order of their creation). 2. Close itself. 3. Throw a `StructureViolationException` to indicate the improper closure sequence. We should instead rely on the `try-with-resource` statement.

Now that we have some understanding of structured concurrency and its API, let's implement the `fetchProductInfo` method using `StructuredTaskScope`. This method will gather product information concurrently by fetching product details and reviews:

```
public ProductInfo fetchProductInfo(Long productId) {
    log("Fetching product & reviews for id: " + productId);
    try (var scope = StructuredTaskScope.open()) { ❶
        StructuredTaskScope.Subtask<Product> productTask =
            scope.fork(() -> fetchProduct(productId)); ❷
        StructuredTaskScope.Subtask<List<Review>> reviewsTask =
            scope.fork(() -> fetchReviews(productId)); ❸
        scope.join(); ❹
        return new ProductInfo(productTask.get(), reviewsTask.get()); ❺
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new ProductServiceException(
            "Fetch failed for id: " + productId); ❻
    }
```



```
}  
}
```

Let's trace through what happens when this structured approach executes:

- ❶ The `fetchProductInfo()` method initiates a `StructuredTaskScope` within a `try-with-resources` block for automatic cleanup.
- ❷ Creates the first subtask to fetch the `Product` and starts it running concurrently.
- ❸ Creates the second subtask to retrieve the `List<Review>` and starts it running concurrently alongside the product task.
- ❹ Calls `scope.join()`, which blocks until both subtasks complete, one fails, or the scope is canceled.
- ❺ If both subtasks succeed, `join()` returns normally, and the method constructs a `ProductInfo` instance using results from `productTask.get()` and `reviewsTask.get()`.
- ❻ If either subtask fails, `join()` interrupts the other running subtask, waits for it to terminate, and throws a `StructuredTaskScope.FailedException` with the original exception as its cause.

NOTE

The structured concurrency feature explored in this chapter is currently available as a preview in JDK 25.

We can enable it using the `--enable-preview` flag during compilation and run-time. To compile and run your code, use the following commands:

- *With javac:* `javac --release 25 --enable-preview Main.java`
- *With java:* `java --enable-preview Main`
- *With the source code launcher:* `java --enable-preview Main.java`
- *With JShell:* `jshell --enable-preview`

We anticipate this feature will soon be integrated into future JDK releases without requiring any modifications to the code.

To see how structured concurrency handles failures, let's modify the `fetchProduct()` method to simulate a scenario where the product is not found:

```
private Product fetchProduct(Long productId) {  
    log("Fetching product id: " + productId);  
    if (productId == 1L) {
```

```

        throw new ProductServiceException("Product not found"); ❶
    }
    sleepForAWhile(Duration.ofSeconds(1)); // Simulate network call
    return new Product(productId, "Sample Product",
        "A great product description.");
}

```

- ❶ Immediately throws an exception for product ID 1 to simulate a “not found” scenario

When we run the program with this modification, the output demonstrates the structured concurrency behavior:

```

04:46:34.321 main : Attempting to fetch product info for ID: 1
04:46:34.321 main : Fetching product & reviews for id: 1
04:46:34.325 VThread[#34]: Fetching product id: 1
04:46:34.325 VThread[#36]: Fetching reviews for id: 1
Exception in thread "main"
    java.util.concurrent.StructuredTaskScope$FailedException:
    ca.bazlur.mcj.chap4.ProductServiceException:
    at java.base/java.util.concurrent.StructuredTaskScopeImpl
        .join(StructuredTaskScopeImpl.java:257)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .fetchProductInfo(FailingProductServiceWithStructuredConcurrency)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .main(FailingProductServiceWithStructuredConcurrency)
Caused by: ca.bazlur.mcj.chap4.ProductServiceException:
    Product not found
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .fetchProduct(FailingProductServiceWithStructuredConcurrency)
    at ca.bazlur.mcj.chap4.FailingProductServiceWithStructuredConcurrency
        .lambda$fetchProductInfo$0(FailingProductServiceWithStructuredConcur
    at java.base/java.util.concurrent.StructuredTaskScopeImpl$SubtaskImpl
        .run(StructuredTaskScopeImpl.java)
    at java.base/java.lang.VirtualThread.run(VirtualThread.java:456)

```

The stack trace reveals exactly what we hoped to see. Because the `fetchProduct()` subtask threw an exception, `StructuredTaskScope` immediately canceled the concurrently running `fetchReviews()` subtask. The `scope.join()` method then threw a `StructuredTaskScope.FailedException`, wrapping the original `ProductServiceException` that occurred in `fetchProduct()` as its cause so that the parent thread can handle the error gracefully ([Figure 4-2](#)).

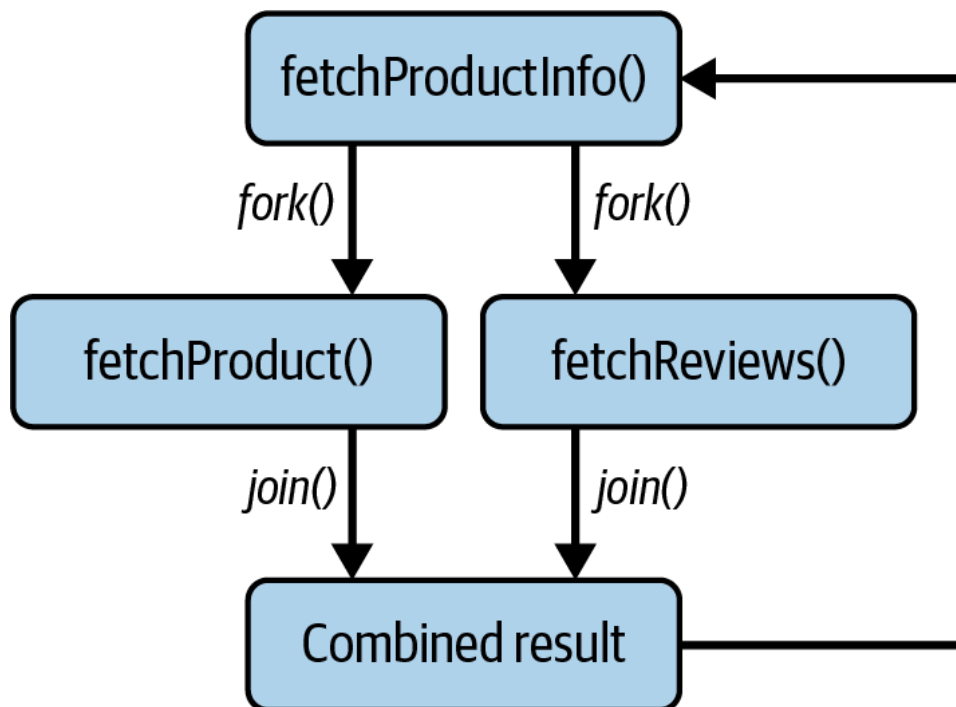


Figure 4-2. A `StructuredTaskScope` workflow where `fetchProductInfo()` forks `fetchProduct()` and `fetchReviews()`, joins their results, and ensures cancellation on failure

This structured version not only addresses the limitations of our previous unstructured approach but also brings significant benefits. If either task fails, the other is canceled automatically, preventing the resource waste we saw earlier. If the parent task is canceled, the scope closes and automatically terminates both subtasks. The `join()` method ensures that both tasks complete before results are retrieved, preventing the incomplete data issues that plagued our original implementation.

Beyond the technical improvements, the structured code is more concise, easier to understand, and less prone to errors. It clearly reflects the intent of the operation: fetching both the product and reviews in parallel, but only proceeding if both succeed. There is no ambiguity about what should happen when things go wrong.

By embracing structured concurrency, we can create concurrent code that is more efficient, reliable, and maintainable. This reliability instills a sense of security, knowing that our code will perform as expected.

Scopes and Subtasks: Relationship and Lifecycle

Now that we understand structured concurrency and its APIs, let's explore the relationship between scopes and their subtasks.

Structured concurrency uses `StructuredTaskScope` as a parent container to manage its child subtasks, handling their lifecycle and execution. Each subtask represents a unit of concurrent work running on virtual threads within the scope. This parent-child relationship ensures a coordinated and predictable execution flow.

The `StructuredTaskScope` defines a bounded context for concurrent operations, managing the entire lifecycle of its subtasks to ensure they execute and terminate in a disciplined manner.

Forking subtasks

To initiate concurrent work, the scope's owner uses the `fork()` method. The API provides two distinct overloads for this purpose:

```
fork(Callable<? extends U> task)
```

This method is used for subtasks that are designed to return a value. It takes a `Callable` and, upon execution in a new virtual thread, its `call()` method will be invoked.

```
fork(Runnable task)
```

This method is used for subtasks that perform an action but do not return a result. It works identically to the `Callable` version, but if the subtask completes successfully, its corresponding `Subtask.get()` method will simply return `null`.

The fork process

Regardless of which `fork()` method is called, the process is the same:

- A `Subtask` object is created to represent the asynchronous task. This `Subtask` acts as a handle.
- An internal `Joiner` policy object is consulted via its `onFork()` method. If the policy determines the subtask should not run (e.g., if the scope is already canceled), `fork()` returns the handle and no new thread is started.
- Otherwise, a new virtual thread is started to execute the task.

The `fork()` method immediately returns the `Subtask` handle. The scope's owner can only use this handle to get the result with `Subtask.get()` or the exception with `Subtask.exception()` after the scope has been joined via the `join()` method.

Subtask completion

When a subtask finishes its work, the thread executing it notifies the joiner by invoking its `onComplete()` method, passing the `Subtask` handle, which now contains the final state (e.g., `SUCCESS`, `UNAVAILABLE`, or `FAILED`). We will discuss `Joiner` in a later section.

The scope owner uses `join()` to wait for the subtask policy to be satisfied. For the default scope created by `StructuredTaskScope.open()`, this means waiting until either all subtasks complete successfully or one subtask fails. If a subtask fails, the scope automatically cancels any other

running subtasks. The `join()` method then throws a `FailedException` that wraps the original exception from the subtask that failed.

Finally, when the scope is closed (either explicitly or via a `try-with-resources` block), it guarantees a clean shutdown by waiting for all subtask threads to terminate before allowing the parent thread to proceed. This prevents resource leaks and ensures the structured nature of the concurrent operations.

Joining Policies with Joiner

A core feature of `StructuredTaskScope` is its ability to define flexible joining policies through a `Joiner` object. A `Joiner` is a powerful mechanism that manages the lifecycle of subtasks by dictating the conditions under which the `join()` method completes and what kind of result, or outcome, it produces.

Instead of a few fixed policies, Java provides the `Joiner` interface. A simplified view of its structure shows both the outcome methods and the lifecycle hooks:

```
public interface Joiner<T, R> {  
    R result(); ❶  
    Throwable exception(); ❷  
    default boolean onFork(Subtask<? extends T> subtask) { ... } ❸  
    default boolean onComplete(Subtask<? extends T> subtask) { ... } ❹  
}
```

Let's look at what this method entails:

- ❶ Outcome method that defines the successful result to return from `scope.join()`
- ❷ Outcome method that defines the exception to throw from `scope.join()`
- ❸ Lifecycle hook called every time `fork()` is invoked to track new subtasks
- ❹ Lifecycle hook called when any subtask finishes, containing the core policy logic

The first two methods, `result()` and `exception()`, are outcome methods that define the contract for the final outcome of `scope.join()`. An implementation must provide logic for one of these to return either a successful value or an exception to be thrown.

The `onFork()` method is a lifecycle hook that allows the `Joiner` to be aware of and track new subtasks as they are created. The `onComplete()` method is where the core policy logic typically resides, checking if the completion of a specific subtask satisfies the overall joining condition (such as “is this the first success?” or “have all tasks now failed?”).

Common Joining Policies

The `Joiner` provides several static factory methods for creating common policies:

`Joiner.awaitAllSuccessfulOrThrow()`

This is the default policy. It is used when we call `StructuredTaskScope.open()`. Its internal `onComplete()` logic tracks the completion count. If a subtask fails, it cancels the scope. It only allows `join()` to complete successfully when all forked subtasks have succeeded.

`Joiner.anySuccessfulResultOrThrow()`

This implements a “race to win” policy. Its `onComplete()` logic checks if the completing subtask was successful. If so, it immediately cancels the scope, and its `result()` method returns the successful result.

`Joiner.allSuccessfulOrThrow()`

This policy’s `onComplete()` hook collects the result of every subtask that completes successfully. It waits for all subtasks to finish and then makes the collected results available in a `Stream`.

`Joiner.awaitAll()`

This policy waits for all subtasks to complete, whether they succeed or fail. It is useful when you need to know the outcome of every task and handle each one individually. The `scope.join()` method returns a `Stream<Subtask>` that you can process to inspect the state, result, or exception of each subtask.

`Joiner.allUntil(Predicate<Subtask> isDone)`

This is the most flexible factory method, allowing for custom cancellation logic. It takes a `Predicate` that is tested against every subtask upon its completion. If the predicate returns true, the scope is canceled. This allows for complex policies, such as “cancel after two subtasks fail.” If the predicate never returns true, it behaves like `awaitAll()`, waiting for all subtasks to finish. In either case, `scope.join()` returns a `Stream<Subtask>` of all subtasks that were forked.

This `Joiner`-based approach allows for fine-grained control over the scope’s completion criteria and its return value, from waiting for all sub-

tasks to succeed, to racing them to find the first available result, to collecting multiple results in a stream.

Let's explore each of these policies with practical examples.

Wait for all to succeed or first to fail

When you create a `StructuredTaskScope` using the `open()` method, it defaults to a policy where all subtasks must succeed. This “all-or-nothing” approach is internally managed by

`Joiner.awaitAllSuccessfulOrThrow()`. This means that if even one subtask fails, the entire scope is shut down. Any remaining, in-progress subtasks are immediately canceled, and the error from the failing subtask is propagated back to the parent task. This design ensures that you only get a result if every part of your operation completes successfully, preventing the use of incomplete or partial results.

Let's illustrate this with a practical example. We will first create data records similar to those we made earlier in the chapter:

```
record Product(long productId, String name) {}
record Review(String user, int rating) {}
record ProductInfo(Product product, List<Review> reviews) {}
```

Next, we need the methods that will run as our concurrent subtasks. To make the concurrency visible, we'll add `log()` statements and use `Thread.sleep()` to simulate network latency. We will also create a specific method, `fetchProductThatFails()`, to demonstrate the error-handling path:

```
// A subtask that succeeds after a 1-second delay
private Product fetchProduct(long productId)
    throws InterruptedException {
    log(" -> Fetching product details... (will take 1s)");
    Thread.sleep(Duration.ofSeconds(1)); ❶
    log(" <- Product details fetched.");
    return new Product(productId, "Sample Product");
}
// A subtask that will always fail
private Product fetchProductThatFails(long productId) {
    log(" -> Fetching product details... (will fail)");
    throw new ProductServiceException("Product ID " + productId + " not found");
}
// A subtask that succeeds after a 2-second delay
private List<Review> fetchReviews(long productId)
    throws InterruptedException {
    log(" -> Fetching product reviews... (will take 2s)");
    Thread.sleep(Duration.ofSeconds(2)); ❸
    log(" <- Product reviews fetched.");
}
```



```

        return List.of(new Review("Inaya", 5), new Review("Rushda", 4));
    }
}

```

Each method serves a specific purpose in our demonstration:

- ❶ Simulates a one-second network call for product data
- ❷ Immediately throws an exception to simulate a “product not found” scenario
- ❸ Simulates a two-second network call for review data.

We defined the `log()` method in the previous section. Let’s convert it into a utility method so that we can reuse it in other contexts:

```

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
public class Utils {
    public static void log(String message) {
        Thread currentThread = Thread.currentThread();
        String threadName = currentThread.isVirtual()
            ? "VThread[#" + currentThread.threadId() + "]"
            : currentThread.getName();
        String currentTime = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
        System.out.printf("%s %-12s: %s%n", currentTime, threadName, message);
    }
}

```

Now let’s define our main logic using the default policy:

```

public ProductInfo fetchProductInfo(long productId, boolean shouldFail)
    throws InterruptedException {
    Instant start = Instant.now();
    // Using open() provides the default "fail-fast" policy
    try (var scope = StructuredTaskScope.open()) {
        StructuredTaskScope.Subtask<Product> productTask = shouldFail
            ? scope.fork(() -> fetchProductThatFails(productId))
            : scope.fork(() -> fetchProduct(productId));
        StructuredTaskScope.Subtask<List<Review>> reviewsTask
            = scope.fork(() -> fetchReviews(productId));
        // Waits for both to succeed, or throws FailedException on first failure
        log("... Scope joining. Waiting for subtasks...");
        scope.join();
        log("... Scope joined successfully.");
        // Only reachable if join() succeeds
        return new ProductInfo(productTask.get(), reviewsTask.get());
    } catch (StructuredTaskScope.FailedException ex) {
        // This block executes only in the failure scenario
        log("... Scope join failed. A subtask threw an exception.");
        throw new RuntimeException("Failed to fetch product info", ex.getCause());
    }
}

```

```

    } finally {
        Instant end = Instant.now();
        log("Total time taken: " + Duration.between(start, end).toMillis() + "ms");
    }
}

```

Let's trace through the execution flow:

- ❶ Creates a scope with the default `awaitAllSuccessfulOrThrow()` policy
- ❷ Conditionally forks a task that will fail (for demonstration)
- ❸ Forks a task that will succeed after one second
- ❹ Forks a task that will succeed after two seconds
- ❺ Waits for all subtasks to complete successfully or fails fast on first error
- ❻ Retrieves results only if all subtasks succeeded
- ❼ Handles the failure case by wrapping the original exception

This is the core of our example. The `fetchProductInfo()` method uses `StructuredTaskScope` to orchestrate the subtasks. A boolean flag, `shouldFail`, will allow us to easily switch between the success and failure paths.

Here's our complete demonstration class:

```

public class DefaultPolicyDemo {
    // ... (method implementations from above)
    void main() {
        var demo = new DefaultPolicyDemo();
        log("--- Running Success Scenario ---");
        log("... Expecting to take ~2 seconds (the time of the slowest task)...");
        try {
            ProductInfo result = demo.fetchProductInfo(123L, false); ❶
            log("\nSuccess! Result: " + result);
        } catch (Exception e) {
            log("\nCaught unexpected exception in success scenario: " +
                e.getMessage());
        }
    }
}

```

The main method demonstrates the success path:

- ❶ Runs the success scenario by passing `false` for the `shouldFail` parameter

Let's run the code:

```
java --enable-preview DefaultPolicyDemo.java
```

NOTE

Please note that we are using JDK 25, and structured concurrency is a preview feature.

The output would be as follows:

```
20:35:18.357 main          : --- Running Success Scenario ---
20:35:18.358 main          : ... Expecting to take ~2 seconds (the time of
slowest task)...
20:35:18.362 main          : ... Scope joining. Waiting for subtasks...
20:35:18.363 VThread[#36]   : -> Fetching product reviews... (will take 2s
20:35:18.363 VThread[#34]   : -> Fetching product details... (will take 1s
20:35:19.367 VThread[#34]   : <- Product details fetched.
20:35:20.369 VThread[#36]   : <- Product reviews fetched.
20:35:20.370 main          : ... Scope joined successfully.
20:35:20.371 main          : Total time taken: 2012ms
20:35:20.378 main          :
Success! Result: ProductInfo[product=Product[productId=123, name=Sample Pro
reviews=[Review[user=Inaya, rating=5], Review[user=Rushda, rating=4]]]
```

The main thread creates the scope and immediately forks two subtasks, `fetchProduct()` and `fetchReviews()`, which are picked up by virtual threads (`VThread[#34]` and `VThread[#36]`) and start executing at nearly the same instant. While they run in parallel, the main thread blocks at the `scope.join()`, waiting for the outcome. After one second, the `fetchProduct()` task completes, but the main thread continues to wait as the slower `fetchReviews()` task is still running. Once `fetchReviews()` completes after two seconds, and now that all subtasks have succeeded, the `join()` method finally returns. The total time taken is ~2,000ms, indicating that the tasks ran in parallel, as the operation's duration is bounded by its longest-running subtask.

Now, let's turn our attention to the failure scenario. To do this, we'll modify the main method to call `fetchProductInfo()` with the `shouldFail` flag set to `true`. This will cause it to use the `fetchProductThatFails()` method, which we've designed to immediately throw an exception.

Next, we'll modify our main method to execute this path:

```
void main() {
    var demo = new DefaultPolicyDemo();
    log("--- Running Failure Scenario ---");
```

```

log("... Expecting to fail almost instantly...\n");
try {
    demo.fetchProductInfo(456L, true); ❶
} catch (Exception e) {
    // We expect the RuntimeException thrown from our catch block
    log("\nCaught expected exception in failure scenario: " + e.getMessage());
    log("Cause: " + e.getCause()); ❷
}
}

```

The failure scenario demonstrates immediate error handling:

- ❶ Runs the failure scenario by passing `true` for the `shouldFail` parameter
- ❷ Shows the original cause of the failure wrapped in our `RuntimeException`

The output would be as follows:

```

22:20:11.405 main          : --- Running Failure Scenario ---
22:20:11.406 main          : ... Expecting to fail almost instantly...
22:20:11.411 main          : ... Scope joining. Waiting for subtasks...
22:20:11.412 VThread[#34]  : -> Fetching product details... (will fail)
22:20:11.412 VThread[#36]  : -> Fetching product reviews... (will take 2s)
22:20:11.413 main          : ... Scope join failed. A subtask threw an exce
22:20:11.414 main          : Total time taken: 7ms
22:20:11.414 main          :
Caught expected exception in failure scenario: Failed to fetch product info
22:20:11.414 main          : Cause: ca.bazlur.mcj.chap4.
ProductServiceException: Product ID 456 not found

```

This output clearly demonstrates the power of the fail-fast policy.

Although both the failing `fetchProductThatFails()` task and the two-second `fetchReviews()` task are forked onto virtual threads, the entire operation is over in just seven milliseconds. The moment the first subtask fails, the `Joiner` policy cancels the other running subtask and causes `scope.join()` to immediately throw a `FailedException`. The system does not waste time waiting for the `fetchReviews()` task to complete, as its result would be irrelevant. This rapid error propagation and prevention of wasted work is a core advantage of using structured concurrency for “all-or-nothing” operations.

To fully appreciate the benefits of the default fail-fast policy, let’s compare it with a traditional `ExecutorService` approach for the same task:

```

ProductInfo fetchProductInfoWithExecutor(Long productId)
    throws ExecutionException, InterruptedException {
    Instant start = Instant.now();

```

```

try (ExecutorService service
    = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<Product> productFuture
        = service.submit(() -> fetchProductThatFails(productId)); ❷
    Future<List<Review>> reviewFuture
        = service.submit(() -> fetchReviews(productId)); ❸
    Product product = productFuture.get(); ❹
    List<Review> reviews = reviewFuture.get(); ❺
    return new ProductInfo(product, reviews);
} finally {
    Instant end = Instant.now();
    System.out.printf("Time taken: %dms%n",
        end.toEpochMilli() - start.toEpochMilli());
}
}

```

The traditional approach shows its limitations:

- ❶ Creates an `ExecutorService` with virtual threads
- ❷ Submits the failing product task and gets a `Future`
- ❸ Submits the reviews task and gets another task
- ❹ Blocking `get` method invocation waiting for the product result (which will fail)
- ❺ Would block waiting for the review results, but is never reached due to an exception

If we call this method from the main method, the output reveals the inefficiency:

```

22:46:10.211 main          : --- Running Failure Scenario with Executor ---
22:46:10.212 main          : ... Expecting to fail almost instantly...
22:46:10.215 VThread[#36]  : -> Fetching product reviews... (will take 2s
22:46:10.215 VThread[#34]  : -> Fetching product details... (will fail)
22:46:12.221 VThread[#36]  : <- Product reviews fetched.
Time taken: 2010ms
22:46:12.227 main          :
Caught expected exception in failure scenario: ca.bazlur.mcj.chap4.Products
Exception: Product ID 456 not found
22:46:12.227 main          : Cause: ca.bazlur.mcj.chap4.ProductServiceExce
Product ID 456 not found

```

When we dissect the console log, the critical inefficiency of this unstructured approach becomes clear. We see that even though our `fetchProductThatFails()` task on `VThread[#34]` fails almost instantly, the `fetchReviews()` task on `VThread[#36]` continues running, completely unaware. The log confirms this by printing the `Product reviews fetched` message after the full two-second delay. This happens because the `ExecutorService` provides no concept of a shared scope;

from its perspective, the two `Future`s are independent, and a failure in one has no bearing on the other. As a result, our program is blocked while the reviews task runs to completion, wasting system resources on a result we will ultimately discard.

Race for the first successful result

With the `Joiner.anySuccessfulResultOrThrow()` policy, we can orchestrate a race among our concurrent subtasks, declaring a winner as soon as one completes successfully. We can think of this as a “first-past-the-post” approach. The moment we have a winner, the `Joiner` cancels all other running subtasks.

To illustrate this policy, we’ll set up a scenario where we fetch product details from three different sources, each with a different response time. We want to proceed with whichever result we get first.

First, we need to define our three `fetch` methods. We’ll give each a different `Thread.sleep()` duration and add `log()` statements. This will allow us to observe the race and the ensuing cancellation in action when we run the code:

```
private Product fetchProductFromCache(long productId) throws
    InterruptedException {
    log(" -> Checking cache... (will take 500ms)");
    Thread.sleep(Duration.ofMillis(500)); ❶
    log(" <- Cache has the result!");
    return new Product(productId, "Product from Cache");
}
// A slower source (2000ms)
private Product fetchProductFromDatabase(long productId)
    throws InterruptedException {
    try {
        log(" -> Querying database... (will take 2s)");
        Thread.sleep(Duration.ofSeconds(2)); ❷
        log(" <- Database has the result!");
        return new Product(productId, "Product from DB");
    } catch (InterruptedException e) {
        // This block will execute when the scope cancels this task
        log(" <- Database query was canceled."); ❸
        Thread.currentThread().interrupt();
        throw e;
    }
}
// The slowest source (3000ms)
private Product fetchProductFromAPI(long productId)
    throws InterruptedException {
    try {
        log(" -> Calling external API... (will take 3s)");
        Thread.sleep(Duration.ofSeconds(3)); ❹
```

```

        log(" <- API has the result!");
        return new Product(productId, "Product from API");
    } catch (InterruptedException e) {
        // This block will also execute upon cancellation
        log(" <- API call was canceled."); ❸
        Thread.currentThread().interrupt();
        throw e;
    }
}

```

Each method simulates a different data source with varying response times:

- ❶ The cache responds fastest at 500ms, making it the likely winner.
- ❷ The database takes two seconds and includes cancellation handling.
- ❸ Logs when the database query gets interrupted due to scope cancellation.
- ❹ The API is slowest at three seconds and also handles cancellation.
- ❺ Logs when the API call gets interrupted due to scope cancellation.

Now, we'll write our orchestrator method. This time, when we open the scope, we'll pass in the `Joiner.anySuccessfulResultOrThrow()` joiner to set up the race:

```

public Product fetchProduct(long productId) {
    Instant start = Instant.now();
    // The Joiner specifies the "race-to-win" policy
    try (var scope
        = StructuredTaskScope.open(
            StructuredTaskScope
                .Joiner.<Product>anySuccessfulResultOrThrow())) { ❶
        scope.fork(() -> fetchProductFromDatabase(productId)); ❷
        scope.fork(() -> fetchProductFromCache(productId)); ❸
        scope.fork(() -> fetchProductFromAPI(productId)); ❹
        // join() now returns the result of the first successful subtask
        return scope.join(); ❺
    } catch (InterruptedException | StructuredTaskScope.FailedException e) {
        // FailedException is thrown if ALL subtasks fail
        throw new RuntimeException(e); ❻
    } finally {
        Instant end = Instant.now();
        log("Total time taken: %dms%n"
            .formatted(Duration.between(start, end).toMillis()));
    }
}

```

The orchestrator method sets up the race conditions:

- ❶ Creates a scope with the race-to-win policy that returns the first successful result
- ❷ Forks the database task (two-second delay)
- ❸ Forks the cache task (500ms delay)—likely winner
- ❹ Forks the API task (three-second delay)
- ❺ Waits for the first successful completion and returns that result
- ❻ Handles the case where all subtasks fail (throws `FailedException`)

When we execute this code, we expect to see the cancellation happen live. Let's call this method from the main method:

```
public class RacePolicyDemo {
    // ... (method implementations from above)
    void main() {
        var demo = new RacePolicyDemo();
        log("--- Running Race Scenario ---");
        log("... Three tasks will race. " +
            "Expecting to finish in ~500ms (the fastest task)...\n");
        try {
            Product winningProduct = demo.fetchProduct(123L); ❶
            log("Race finished! Winning result: " + winningProduct);
        } catch (Exception e) {
            log("Caught unexpected exception: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

The main method demonstrates the race scenario:

- ❶ Starts the race between three different data sources and expects the fastest to win

For this code to work we need the following record as well:

```
public record Product(long productId, String source) {
}
```

Let's execute the code:

```
java --enable-preview RacePolicyDemo.java
```

The output will be as follows:

```

23:03:23.917 main      : --- Running Race Scenario ---
23:03:23.917 main      : ... Three tasks will race. Expecting to finish i
~500ms (the fastest task)...
23:03:23.923 VThread[#36]: -> Checking cache... (will take 500ms)
23:03:23.923 VThread[#38]: -> Calling external API... (will take 3s)
23:03:23.923 VThread[#34]: -> Querying database... (will take 2s)
23:03:24.429 VThread[#36]: <- Cache has the result!
23:03:24.431 VThread[#34]: <- Database query was canceled.
23:03:24.431 VThread[#38]: <- API call was canceled.
23:03:24.439 main      : Total time taken: 513ms
23:03:24.442 main      : Race finished! Winning result: Product[productId
source=Product from Cache]

```

When we look at our console output, we can see a perfect demonstration of the race-to-win policy. We see that all three of our `fetch` methods are forked and start running in parallel. However, after about 500ms, our fastest task, `fetchProductFromCache()`, succeeds. At that exact moment, our `Joiner` policy considers its condition met, and we see it immediately cancel the other two running subtasks; we have proof of this in the `was canceled` messages that get printed to the error stream. Our `join` method then returns the winning result from the cache, and our entire operation concludes in just over 500ms, without us having to waste time waiting for the slower tasks.

Gather all results or fail fast

We’ve seen the default policy, which fails if any subtask fails. This next policy, `allSuccessfulOrThrow`, behaves identically in the case of a failure, but offers a more convenient way to handle results upon success.

Like the default policy, `allSuccessfulOrThrow()` maintains a strict “all-or-nothing” strategy. If any subtask fails, it promptly cancels all other running subtasks, and the `scope.join()` method throws a `FailedException`.

The key difference is what happens on success. Instead of `join()` returning `void`, it returns a `java.util.Stream<Subtask<T>>`. This stream contains the handles to all the successfully completed subtasks, in the order they were forked, making it easy to process all the results at once. We should use this `Joiner` when all of our subtasks return the same result type (e.g., `String`, `Product`, etc.) and we need all of them to succeed.

The default `awaitAllSuccessfulOrThrow()` policy is better suited for when our subtasks return results of different types, since we would handle each subtask’s result individually anyway.

Let's imagine we need to validate a list of user IDs by checking them against an external service. We want to do this concurrently, but the entire batch is only valid if every user ID is successfully validated.

First, we'll define a simple method that simulates validating a user. We'll also create a version that fails for a specific ID:

```
public record ValidatedUser(long userId, String status) {
}

private ValidatedUser validateUser(long userId)
    throws InterruptedException {
    log(" -> Validating user %d..." .formatted(userId));
    Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(500))); ❶
    log(" <- User %d is valid." .formatted(userId));
    return new ValidatedUser(userId, "VALID");
}

private ValidatedUser validateUserWithFailure(long userId)
    throws InterruptedException {
    if (userId == 3L) {
        log(" -> Validating user %d... (will fail)" .formatted(userId));
        throw new IllegalArgumentException("Invalid user ID: " + userId); ❷
    }
    return validateUser(userId);
}
```

The validation methods simulate real-world behavior:

- ❶ Adds random delay between 100–600ms to simulate network variability
- ❷ Specifically fails for user ID 3 to demonstrate error handling

Now, we'll write a method that takes a list of user IDs, forks a validation task for each, and uses `allSuccessfulOrThrow()` to collect the results:

```
public List<ValidatedUser> validateAllUsers(List<Long> userIds)
    throws InterruptedException {
    log("Validating a batch of " + userIds.size() + " users...");
    try (var scope
        = open(Joiner.<ValidatedUser>allSuccessfulOrThrow())) { ❶
        var subtasks = userIds.stream()
            .map(id -> scope.fork(() -> validateUserWithFailure(id))) ❷
            .toList();
        // join() returns a Stream<Subtask<ValidatedUser>> on success
        // or throws FailedException on the first failure
        var resultStream = scope.join(); ❸
        log("...All users validated successfully. Processing stream...");
        return resultStream
            .map(Subtask::get) ❹
    }
```

```

        .toList();
    } catch (FailedException ex) {
        log("All users validated successfully. Processing stream");
        throw new RuntimeException("Batch validation failed",
            ex.getCause()); ❸
    }
}

```

The batch validation method demonstrates collecting multiple results:

- ❶ Creates a scope with the `allSuccessfulOrThrow()` policy for collecting all results
- ❷ Forks a validation subtask for each user ID and collects the subtask handles
- ❸ Waits for all validations to complete and returns a `Stream` of successful subtasks
- ❹ Extracts the actual `ValidatedUser` results from each subtask
- ❺ Handles batch failure by wrapping the original validation exception

A NOTE ON VAR AND TYPE INFERENCE

To keep our code concise, we've used the `var` keyword. While this improves readability by reducing boilerplate, it's helpful for us to know the explicit types being used.

In the `validateAllUsers` method:

- `var scope` is inferred as `StructuredTaskScope<ValidatedUser>`
 - `var subtasks` is inferred as `List<Subtask<ValidatedUser>>`
 - `var resultStream` is inferred as `Stream<Subtask<ValidatedUser>>`
-

Now, let's put everything together into our complete `BatchValidationDemo.java` class. To maintain the conciseness of the code we present, we will use static imports for the nested classes of `StructuredTaskScope`, such as `Joiner` and `Subtask`, as well as for the `open()` method itself:

```

import static java.util.concurrent.StructuredTaskScope.*;
public class BatchValidationDemo {
    // ... (method implementations from above)
    void main() {
        var demo = new BatchValidationDemo();
        List<Long> successfulBatch = List.of(1L, 2L, 4L, 5L);
        // Contains the failing ID '3'
        List<Long> failingBatch = List.of(1L, 2L, 3L, 4L, 5L); ❶
        log("--- Running Success Scenario ---");
    }
}

```

```

    try {
        List<ValidatedUser> results = demo.validateAllUsers(successfulBatch);
        log("\nBatch validation complete. Results:");
        results.forEach(validatedUser -> log(validatedUser.toString()));
    } catch (Exception e) {
        log("Caught unexpected exception: " + e.getMessage());
    }
    log("\n=====");
    log("--- Running Failure Scenario ---");
    try {
        demo.validateAllUsers(failingBatch); ❸
    } catch (Exception e) {
        log("Caught expected exception: " + e.getMessage());
    }
}
}

```

The main method tests both scenarios:

- ❶ Creates a batch containing user ID 3, which will trigger our failure condition
- ❷ Tests the success scenario with user IDs that will all validate successfully
- ❸ Tests the failure scenario where one validation fails and cancels the others

Now, let's run the code:

```
java --enable-preview BatchValidationDemo.java
```

The output would be as follows:

```

00:29:49.275 main      : --- Running Success Scenario ---
00:29:49.276 main      : Validating a batch of 4 users...
00:29:49.286 VThread[#39]: -> Validating user 4...
00:29:49.286 VThread[#36]: -> Validating user 2...
00:29:49.286 VThread[#34]: -> Validating user 1...
00:29:49.286 VThread[#42]: -> Validating user 5...
00:29:49.452 VThread[#39]: <- User 4 is valid.
00:29:49.480 VThread[#42]: <- User 5 is valid.
00:29:49.614 VThread[#34]: <- User 1 is valid.
00:29:49.696 VThread[#36]: <- User 2 is valid.
00:29:49.697 main      : All users validated successfully. Processing str
00:29:49.698 main      :
Batch validation complete. Results:
00:29:49.701 main      : ValidatedUser[userId=1, status=VALID]
00:29:49.702 main      : ValidatedUser[userId=2, status=VALID]
00:29:49.702 main      : ValidatedUser[userId=4, status=VALID]
00:29:49.702 main      : ValidatedUser[userId=5, status=VALID]

```

```

00:29:49.702 main      :
=====
00:29:49.702 main      : --- Running Failure Scenario ---
00:29:49.702 main      : Validating a batch of 5 users...
00:29:49.703 VThread[#48]: -> Validating user 4...
00:29:49.703 VThread[#47]: -> Validating user 3... (will fail)
00:29:49.703 VThread[#45]: -> Validating user 1...
00:29:49.703 VThread[#46]: -> Validating user 2...
00:29:49.703 VThread[#49]: -> Validating user 5...
00:29:49.704 main      : ...Validation failed for one of the users.
00:29:49.704 main      : Caught expected exception: Batch validation fail

```

When we assess the output, we can clearly observe the two distinct outcomes of our “all-or-nothing” policy.

In the success scenario, all four validation tasks are forked onto virtual threads and run concurrently. The `main` thread waits at `join()` until the last task completes, after which `join()` returns the `Stream` of results, which we then print.

In the failure scenario, all five tasks are forked, but the task for `userId=3` immediately fails. The `Joiner` detects this failure and cancels the entire scope. The `join()` call never returns a `Stream`; instead, it immediately throws a `FailedException`. Our `catch` block catches this exception and prints the final error message. The key observation is that none of the other tasks print a `<- User X is valid` message, because they were all canceled before they could complete. This confirms the fail-fast behavior and demonstrates how the policy protects the integrity of our batch operation.

Wait for all

We’ve seen policies that either fail fast or require all tasks to succeed. The `awaitAll()` policy takes a different approach: it waits for all subtasks to complete, regardless of whether they succeed or fail. This makes it ideal for scenarios where we care about side effects or need to process all results, even partial ones.

Unlike `allSuccessfulOrThrow()`, the `awaitAll()` policy never cancels running subtasks when one fails. All forked tasks are allowed to run to completion. The `scope.join()` method always returns `null`, since this joiner focuses on task completion rather than result collection.

The key advantage is resilience. When some operations can fail but we still want the benefit of processing successful ones, `awaitAll()` ensures no work is wasted. We should use this joiner when our subtasks perform side effects (like logging, metrics collection, or file processing) or when

we need to gather as much data as possible, even if some sources are unavailable.

The `awaitAll()` policy is particularly useful for monitoring systems, batch processing jobs, and fan-in server architectures where partial success is acceptable and valuable.

Let's imagine we need to send notifications about a critical system event to multiple channels (email, SMS, push notifications). We want maximum reach, so even if one channel fails, the others should still deliver the notification.

First, we'll define a simple record to track notification results, along with methods that simulate sending notifications with varying reliability:

```
public record NotificationResult(String channel, boolean success,
                                String message) {
    public static NotificationResult success(String channel, String message) {
        return new NotificationResult(channel, true, message);
    }

    public static NotificationResult failure(String channel, String error) {
        return new NotificationResult(channel, false, error);
    }
}

// Shared state to collect results (side effects)
private final List<NotificationResult> notificationResults =
    new CopyOnWriteArrayList<>(); ❶
private final AtomicInteger successCount = new AtomicInteger(0); ❷
private final AtomicInteger failureCount = new AtomicInteger(0); ❸
```

The shared state tracks notification outcomes:

- ❶ Thread-safe list to collect results from concurrent notification attempts
- ❷ Atomic counter for successful deliveries across all channels
- ❸ Atomic counter for failed deliveries to track overall reliability

Now let's implement the notification methods with varying reliability:

```
private void sendEmailNotification(String message)
    throws InterruptedException {
    log(" -> Sending email notification...");
    Thread.sleep(Duration.ofMillis(200 + new Random().nextInt(300)));
    // Email is generally reliable (90% success rate)
    if (new Random().nextDouble() < 0.9) { ❶
        log(" <- Email sent successfully");
        notificationResults.add(
            NotificationResult.success("EMAIL", "Delivered to inbox"));
    }
```



```

        successCount.incrementAndGet();
    } else {
        log(" <- Email failed: SMTP server unavailable");
        notificationResults.add(
            NotificationResult.failure("EMAIL", "SMTP server unavailable"));
        failureCount.incrementAndGet();
        throw new RuntimeException("Email delivery failed"); ❷
    }
}

private void sendSmsNotification(String message)
    throws InterruptedException {
    log(" -> Sending SMS notification...");
    Thread.sleep(Duration.ofMillis(150 + new Random().nextInt(400)));
    // SMS is less reliable (70% success rate)
    if (new Random().nextDouble() < 0.7) { ❸
        log(" <- SMS sent successfully");
        notificationResults.add(
            NotificationResult.success("SMS", "Delivered to mobile"));
        successCount.incrementAndGet();
    } else {
        log(" <- SMS failed: Carrier gateway timeout");
        notificationResults.add(
            NotificationResult.failure("SMS", "Carrier gateway timeout"));
        failureCount.incrementAndGet();
        throw new RuntimeException("SMS delivery failed");
    }
}

private void sendPushNotification(String message)
    throws InterruptedException {
    log(" -> Sending push notification...");
    Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(200)));
    // Push notifications are most reliable (95% success rate)
    if (new Random().nextDouble() < 0.95) { ❹
        log(" <- Push notification sent successfully");
        notificationResults.add(
            NotificationResult.success("PUSH", "Delivered to device"));
        successCount.incrementAndGet();
    } else {
        log(" <- Push notification failed: Device token expired");
        notificationResults.add(
            NotificationResult.failure("PUSH", "Device token expired"));
        failureCount.incrementAndGet();
        throw new RuntimeException("Push notification delivery failed");
    }
}

```

Each notification method simulates real-world reliability patterns:

- ❶ Email has a 90% success rate, simulating typical SMTP reliability.
- ❷ Even on failure, we record the result before throwing an exception.

- ③ SMS has a 70% success rate, reflecting carrier gateway challenges.
- ④ Push notifications have a 95% success rate and are the most reliable channel.

Now, we'll write a method that sends notifications to all channels concurrently using `awaitAll()` to ensure all channels are attempted, regardless of individual failures:

```
public void sendCriticalAlert(String alertMessage) throws InterruptedException {
    log("Sending critical alert: " + alertMessage);

    try (var scope = open(Joiner.<Void>awaitAll())) { ❶
        // Fork notification tasks - each performs side effects
        scope.fork(() -> {
            sendEmailNotification(alertMessage);
            return null; // awaitAll() ignores return values ❷
        });
        scope.fork(() -> {
            sendSmsNotification(alertMessage);
            return null;
        });
        scope.fork(() -> {
            sendPushNotification(alertMessage);
            return null;
        });
        log("Waiting for all notification attempts to complete");
        // join() always returns null for awaitAll()
        // All tasks complete regardless of individual failures
        Void result = scope.join(); ❸
        log("...All notification attempts completed.");
        // Process the side effects (collected results)
        logNotificationSummary(); ❹
    } catch (InterruptedException e) {
        log("...Notification sending was interrupted");
        Thread.currentThread().interrupt();
        throw e;
    }
}
```

The alert method demonstrates the `awaitAll()` pattern:

- ❶ Creates a scope with an `awaitAll()` policy that waits for all tasks regardless of failures.
- ❷ Returns `null` since `awaitAll()` focuses on completion, not result collection.
- ❸ `join()` always returns `null` but ensures all tasks have completed.
- ❹ Processes the side effects collected during task execution.

Let's add the summary method that processes our collected results:

```
private void logNotificationSummary() {
    log("\n--- Notification Summary ---");
    log("Total channels attempted: " + (successCount.get() + failureCount.get()));
    log("Successful deliveries: " + successCount.get());
    log("Failed deliveries: " + failureCount.get());
    log("\nDetailed results:");
    notificationResults.forEach(result -> {
        String status = result.success() ? "✅" : "❌";
        log(status + " " + result.channel() + ": " + result.message());
    });
    if (successCount.get() > 0) {
        log("\n📢 Alert successfully delivered through " +
            successCount.get() + " channel(s)");
    } else {
        log("\n⚠️ Alert failed to deliver through any channel!");
    }
}
```

A NOTE ON VAR AND TYPE INFERENCE, PART 2

In the `sendCriticalAlert` method:

- `var scope` is inferred as `StructuredTaskScope<Void>`
- `open()` is statically imported: `import static java.util.concurrent.StructuredTaskScope.open;`

Now, let's put everything together into our complete

`AwaitedAllDemo.java` class. To keep the code we present concise, we will use static imports for `StructuredTaskScope`'s nested classes, like `Joiner`, and the `open()` method itself:

```
import module java.base;
import static ca.bazlur.mcj.chap4.Utills.log;
import static java.util.concurrent.StructuredTaskScope.*;
import static java.util.concurrent.StructuredTaskScope.open;

public class AwaitedAllDemo {
    public record NotificationResult(String channel,
                                     boolean success,
                                     String message) {
        // factory methods and implementation shown above
    }

    private final List<NotificationResult> notificationResults =
        new CopyOnWriteArrayList<>();
    private final AtomicInteger successCount = new AtomicInteger(0);
    private final AtomicInteger failureCount = new AtomicInteger(0);
```

```

// ... (all method implementations from above)
void main() {
    String criticalAlert =
        "URGENT: Database connection pool exhausted - " +
        "immediate attention required";
    log("Running Notification Scenario (awaitAll Policy)");
    log("This demonstrates how awaitAll() processes " +
        "ALL tasks regardless of failures\n");

    try {
        sendCriticalAlert(criticalAlert); ❶
    } catch (Exception e) {
        log("Caught exception: " + e.getMessage());
    }

    log("\n=====");
    log("--- Running Second Notification Batch ---");

    try {
        sendCriticalAlert(
            "RESOLVED: Database issue fixed - " +
            "all systems operational"); ❷
    } catch (Exception e) {
        log("Caught exception: " + e.getMessage());
    }
}
}

```

The `main` method demonstrates the resilience of `awaitAll()`:

- ❶ Sends the first critical alert, allowing us to see how different channels perform
- ❷ Sends a follow-up alert, demonstrating that the pattern works consistently across multiple batches

The `import module java.base;` statement is a module import declaration introduced in JDK 25. It imports, on demand, all public top-level classes and interfaces from:

- The packages exported by the specified module to the current module
- The packages exported by modules that are transitively read due to reading the specified module

For `import module java.base;`, this has the same effect as 54 on-demand package imports (one for each package exported by `java.base`), equivalent to writing `import java.io.*`, `import java.util.*`, `import java.time.*`, `import java.util.concurrent.*`, and so on. This feature eliminates the need for multiple individual package imports when working with fundamental Java APIs, making our code significantly more concise.

Now, let's run the code:

```
java --enable-preview AwaitedAllDemo.java
```

The output would be as follows:

```
04:59:39.852 main      : Running Notification Scenario (awaitAll Policy)
04:59:39.852 main      : This demonstrates how awaitAll() processes ALL t
regardless of failures

04:59:39.852 main      : Sending critical alert to all notification chann
04:59:39.853 main      : Alert message: URGENT: Database connection pool
exhausted - immediate attention required
04:59:39.857 main      : Waiting for all notification attempts to complet
04:59:39.857 VThread[#38]: -> Sending push notification...
04:59:39.857 VThread[#36]: -> Sending SMS notification...
04:59:39.857 VThread[#34]: -> Sending email notification...
04:59:39.970 VThread[#38]: <- Push notification sent successfully
04:59:40.088 VThread[#36]: <- SMS failed: Carrier gateway timeout
04:59:40.234 VThread[#34]: <- Email sent successfully
04:59:40.235 main      : ...All notification attempts completed.
04:59:40.236 main      :
--- Notification Summary ---
04:59:40.237 main      : Total channels attempted: 3
04:59:40.238 main      : Successful deliveries: 2
04:59:40.239 main      : Failed deliveries: 1
04:59:40.239 main      :
Detailed results:
04:59:40.243 main      : ✅ PUSH: Delivered to device
04:59:40.244 main      : ❌ SMS: Carrier gateway timeout
04:59:40.244 main      : ✅ EMAIL: Delivered to inbox
```

```
04:59:40.244 main :  
🌟 Alert successfully delivered through 2 channel(s)
```

When we analyze the output, we can clearly see the resilient behavior of the `awaitAll()` policy.

In this scenario, all three notification tasks are forked onto virtual threads and run concurrently. Even though the SMS notification fails with a `Carrier gateway timeout` error, the email and push notifications continue running and complete successfully. The main thread waits at `join()` until all tasks finish, regardless of their individual outcomes. The side effects (notification results) are collected and summarized, showing that two out of three channels successfully delivered the alert.

The key observation is that unlike `allSuccessfulOrThrow()`, failed tasks never cancel other running tasks. Each notification channel gets a fair chance to deliver the message, maximizing our reach. This behavior is crucial for critical alerting systems where partial delivery is infinitely better than no delivery at all. The policy ensures that temporary issues with one channel don't prevent successful delivery through other available channels.

Resilient concurrent server

Another compelling use case for `awaitAll()` is building resilient concurrent servers. In such servers, you want to ensure that every client connection is handled and allowed to complete, even if one specific connection handler encounters an error. The `awaitAll()` policy prevents one failing client from crashing or prematurely stopping the server's ability to handle other active connections. Unlike `allSuccessfulOrThrow()`, which would cancel all active connections when one fails, `awaitAll()` ensures that each client connection runs to completion independently.

The key advantage is fault isolation. When individual connection handlers encounter errors (network timeouts, malformed requests, client disconnections), these failures remain isolated and don't propagate to other active connections. This makes `awaitAll()` the ideal choice for server applications where uptime and reliability are paramount.

Let's examine a resilient echo server that demonstrates this pattern. We'll start with the main server infrastructure and connection tracking:

```
import module java.base;  
import static ca.bazlur.mcj.chap4.rewrite.Uutils.log;  
import static java.util.concurrent.StructuredTaskScope.open;  
  
public class ResilientServer {
```

```

        private final AtomicInteger connectionCount = new AtomicInteger(0);
        private final AtomicInteger activeConnections = new AtomicInteger(0);
    }

```

These atomic counters will allow us to safely track both the total number of connections received and the number of currently active connections across multiple threads.

Now let's write the core server method that demonstrates how `awaitAll()` manages multiple concurrent connections:

```

public void serve(ServerSocket serverSocket)
    throws IOException, InterruptedException {
    log("Server starting on port: " +
        serverSocket.getLocalPort());

    try (var scope = open(StructuredTaskScope.Joiner.
        <Void>awaitAll())) {

        serverSocket.setSoTimeout(1000);

        while (!Thread.currentThread().isInterrupted()) {
            try {
                Socket socket = serverSocket.accept();
                int connId = connectionCount.incrementAndGet();
                activeConnections.incrementAndGet();

                log("Accepted connection #" + connId);

                // Fork a task to handle this connection
                scope.fork(() -> {
                    handleConnection(socket, connId);
                    return null;
                });

            } catch (SocketTimeoutException e) {
                continue; // Check for interruption
            }
        }

        log("Server stopping, waiting for " +
            "connections to finish...");
        scope.join(); // Wait for all connections to complete

    } finally {
        if (!serverSocket.isClosed()) {
            serverSocket.close();
        }
        log("Server shutdown complete. Total connections: " +
            connectionCount.get());
    }
}

```



```

    }
}

```

The connection handler demonstrates error isolation:

```

private void handleConnection(Socket socket, int connectionId) {
    try (socket;
        var reader = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        var writer = new PrintWriter(
            socket.getOutputStream(), true)) {

        log(" [Conn-" + connectionId + "] Started");
        writer.println("Welcome to Echo Server! " +
            "Type 'quit' to exit.");

        String line;
        while ((line = reader.readLine()) != null) {
            log(" [Conn-" + connectionId + "] Received: " + line);

            if ("quit".equalsIgnoreCase(line.trim())) {
                writer.println("Goodbye!");
                break;
            }

            // Echo back the message
            writer.println("Echo: " + line);
        }

        log(" [Conn-" + connectionId + "] Completed successfully");

    } catch (IOException e) {
        log(" [Conn-" + connectionId + "] Error: " +
            e.getMessage());
    } finally {
        activeConnections.decrementAndGet();
        log(" [Conn-" + connectionId + "] Finished. Active: " +
            activeConnections.get());
    }
}

```

Each client connection is handled in its own forked task. When one connection encounters an error, it only affects that specific connection—other active connections continue processing normally. `IOException` in one connection handler doesn't cancel other running connections. Each connection completes at its own pace.

Each connection handler properly closes its resources in the `finally` block, preventing resource leaks even when errors occur.

Let's run the server and observe its resilient behavior:

```
java --enable-preview ResilientServer.java
```

Now connect this server from two different terminals:

Terminal # 1

```
telnet localhost 8080  
Trying ::1...  
Connected to localhost.  
Escape character is '^]'.  
Welcome to Echo Server! Type 'quit' to exit.  
hello  
Echo: hello
```

Terminal # 2

```
telnet localhost 8080  
Trying ::1...  
Connected to localhost.  
Escape character is '^]'.  
Welcome to Echo Server! Type 'quit' to exit.  
hello  
Echo: hello
```

The output of the server would look as follows:

```
06:16:05.611 main      : Server starting on port: 8080  
06:16:10.631 main      : Accepted connection #1  
06:16:10.641 VThread[#34]: [Conn-1] Started  
06:16:23.630 VThread[#34]: [Conn-1] Received: hello  
06:16:33.990 main      : Accepted connection #2  
06:16:33.991 VThread[#40]: [Conn-2] Started  
06:16:35.973 VThread[#40]: [Conn-2] Received: hello  
06:16:37.549 VThread[#40]: [Conn-2] Received: quit  
06:16:37.553 VThread[#40]: [Conn-2] Completed successfully  
06:16:37.556 VThread[#40]: [Conn-2] Finished. Active: 1  
06:16:41.799 VThread[#34]: [Conn-1] Received: quit  
06:16:41.801 VThread[#34]: [Conn-1] Completed successfully  
06:16:41.802 VThread[#34]: [Conn-1] Finished. Active: 0
```

When we analyze this output, we can see how `awaitAll()` allows each connection to complete independently at its own pace. This demonstrates

the key benefit of `awaitAll()` for server applications: concurrent connections operate without interfering with each other, making the server both resilient and efficient.

After first success

The `Joiner.allUntil(Predicate<Subtask> isDone)` method lets you define custom stopping conditions. Let's create a backup service that stops as soon as any backup location succeeds.

First, we'll define our result record and shared state:

```
public record BackupResult(String location, boolean success) {}
private final AtomicBoolean hasSuccess = new AtomicBoolean(false);
```

Now let's implement our backup methods:

```
private BackupResult backupToCloud(String data) throws InterruptedException {
    log(" -> Backing up to cloud...");
    Thread.sleep(Duration.ofMillis(500));

    if (new Random().nextBoolean()) { ❶
        log(" <- Cloud backup successful");
        hasSuccess.set(true);
        return new BackupResult("Cloud", true);
    } else {
        log(" <- Cloud backup failed");
        return new BackupResult("Cloud", false);
    }
}

private BackupResult backupToUSB(String data) throws InterruptedException {
    log(" -> Backing up to USB...");
    Thread.sleep(Duration.ofMillis(300));

    if (new Random().nextBoolean()) { ❷
        log(" <- USB backup successful");
        hasSuccess.set(true);
        return new BackupResult("USB", true);
    } else {
        log(" <- USB backup failed");
        return new BackupResult("USB", false);
    }
}

private BackupResult backupToNetwork(String data) throws InterruptedException {
    log(" -> Backing up to network drive...");
    Thread.sleep(Duration.ofMillis(400));

    if (new Random().nextBoolean()) { ❸
        log(" <- Network backup successful");
```

```

        hasSuccess.set(true);
        return new BackupResult("Network", true);
    } else {
        log(" <- Network backup failed");
        return new BackupResult("Network", false);
    }
}

```

Each backup method simulates different scenarios:

- ❶ Cloud backup has a 50% success rate.
- ❷ USB backup has a 50% success rate.
- ❸ Network backup has a 50% success rate.

Now let's implement the backup orchestration using `allUntil()`:

```

public void performBackup(String data) throws InterruptedException {
    log("Starting backup to multiple locations...");

    try (var scope = open(Joiner.<BackupResult>allUntil(subtask -> {
        boolean shouldStop = hasSuccess.get(); ❶
        if (shouldStop) {
            log("✅ Backup successful! Canceling other attempts...");
        }
        return shouldStop;
    }))) {

        scope.fork(() -> backupToCloud(data)); ❷
        scope.fork(() -> backupToUSB(data)); ❸
        scope.fork(() -> backupToNetwork(data)); ❹

        scope.join(); ❺

        if (hasSuccess.get()) {
            log("Backup completed successfully!");
        } else {
            log("All backup attempts failed!");
        }
    }
}

```

The backup orchestration illustrates stopping upon the first success:

- ❶ The predicate stops execution as soon as any backup is successful.
- ❷ It attempts a cloud backup.
- ❸ It attempts a USB backup.
- ❹ It attempts a network backup.
- ❺ It waits for the first success or for all backups to fail.

Complete demonstration class:

```
import static java.util.concurrent.StructuredTaskScope.*;

public class BackupDemo {
    // ... (methods from above)

    void main() {
        try {
            performBackup("important-data.zip");
        } catch (Exception e) {
            log("Error: " + e.getMessage());
        }
    }
}
```

Exception Handling in StructuredTaskScope

Exception handling in structured concurrency adheres to well-defined patterns that depend on the `Joiner` policy in use. A

`StructuredTaskScope` is initiated with a `Joiner` that manages sub-task completion and produces the outcome for the `join` method. In certain instances, the outcome will be a result, while in others, it will be an exception.

When the outcome is an exception, the `join` method throws `StructuredTaskScope.FailedException` with the original exception as the cause. For many `Joiner` implementations, this exception will be from a subtask that failed. In the case of `allSuccessfulOrThrow()` and `awaitAllSuccessfulOrThrow()`, for example, the exception is from the first subtask to fail.

Many of the details regarding how exceptions are handled will depend on usage. In some cases, it may be beneficial to add a `catch` block to the `try-with-resources` statement to catch `FailedException`. In other cases, it may be more advantageous to allow `FailedException` to propagate to higher levels for centralized error handling.

Let's explore the different exception-handling strategies through practical examples, starting with the most straightforward approach and building toward more sophisticated patterns.

Basic exception handling

The most common pattern is to wrap your structured concurrency code in a `try-catch` block that handles `FailedException`. This allows you to examine the underlying cause and decide how to respond.

Consider a user data service that needs to fetch information from multiple sources. If any source fails, we want to provide a meaningful error message rather than exposing internal system details:

```
import module java.base;
import static ca.bazlur.mcj.Utls.log;
import static java.util.concurrent.StructuredTaskScope.*;

public class BasicExceptionHandling {

    public String fetchUserData(String userId) {
        try (var scope = open(Joiner.<String>allSuccessfulOrThrow())) { ❶
            var profileTask = scope.fork(() -> fetchUserProfile(userId));
            var preferencesTask = scope.fork(() ->
                fetchUserPreferences(userId));

            var results = scope.join();

            // Process successful results
            return results.map(Subtask::get)
                .collect(Collectors.joining(", "));

        } catch (FailedException e) { ❷
            log("Task failed: " + e.getCause().getMessage());
            return "Error: Unable to fetch user data";
        } catch (InterruptedException e) { ❸
            Thread.currentThread().interrupt();
            throw new RuntimeException("Operation interrupted", e);
        }
    }

    private String fetchUserProfile(String userId)
        throws InterruptedException {
        Thread.sleep(Duration.ofMillis(200));
        if ("invalid".equals(userId)) {
            throw new IllegalArgumentException("Invalid user ID");
        }
        return "Profile for " + userId;
    }

    private String fetchUserPreferences(String userId)
        throws InterruptedException {
        Thread.sleep(Duration.ofMillis(150));
        if (userId.startsWith("blocked")) {
            throw new SecurityException("User access blocked");
        }
        return "Preferences for " + userId;
    }

    void main() {
        var demo = new BasicExceptionHandling();
        demo.fetchUserData("invalid");
    }
}
```

```
}  
}
```

Let's break down what's in this code:

- ❶ The `allSuccessfulOrThrow()` joiner ensures that if any subtask fails, the entire operation fails with a `FailedException`.
- ❷ Use `getCause()` to access the original exception wrapped inside `FailedException` for logging purposes.
- ❸ Always restore the interrupted status when handling `InterruptedException`.

In this example, we use `allSuccessfulOrThrow()` because we need both the user profile and preferences to provide meaningful data. If either subtask fails, the entire operation should fail, but we want to present a user-friendly error message rather than exposing the technical details.

Pattern matching for sophisticated error handling

When you need to handle different types of exceptions with specific recovery strategies, pattern matching provides an elegant solution. This approach enables you to customize your response according to the type of failure.

Let's consider a real-world example that every developer can relate to: an ecommerce order-processing system. When a customer places an order, several operations must succeed: payment processing, inventory verification, and shipping calculation. Each can fail in different ways, and we want to provide specific, actionable feedback to help customers understand what went wrong.

In traditional exception handling, you might catch a generic exception and return a vague "something went wrong" message. With pattern matching in structured concurrency, we can offer precise, helpful responses that guide customers toward resolution.

Consider the following code snippets:

```
import module java.base;  
import static java.util.concurrent.StructuredTaskScope.*;  
import static java.util.concurrent.StructuredTaskScope.open;  
  
public class OrderProcessingService {  
    public record OrderResult(String orderId,  
                              String status,  
                              String message,  
                              boolean successful) {
```

```

    }
    public OrderResult processOrder(String customerId,
                                    String productId, double amount) {
        try (var scope = open(Joiner.<String>allSuccessfulOrThrow())) { ❶
            var paymentTask = scope.fork(() ->
                processPayment(customerId, amount));
            var inventoryTask = scope.fork(() ->
                checkAndReserveInventory(productId));
            var shippingTask = scope.fork(() ->
                calculateShipping(customerId, productId));
            var results = scope.join()
                .map(Subtask::get)
                .toList();
            String orderId = generateOrderId();
            return new OrderResult(orderId, "CONFIRMED",
                "Order confirmed successfully", true);
        } catch (FailedException e) {
            Throwable cause = e.getCause(); ❷
            return handleOrderProcessingError(cause);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new RuntimeException("Operation interrupted", e);
        }
    }

    private static OrderResult handleOrderProcessingError(Throwable cause)
    return switch (cause) { ❸
        case PaymentDeclinedException pde -> new OrderResult(null,
            "PAYMENT_FAILED",
            ""
            "Your payment was declined. Please check your
            card details or try a different payment method.",
            false);
        case InsufficientInventoryException iie ->
            new OrderResult(null, "OUT_OF_STOCK",
                ""
                "Sorry, this item is currently out of stock.
                We'll notify you when it becomes available.",
                false);
        case ShippingNotAvailableException snae ->
            new OrderResult(null, "SHIPPING_UNAVAILABLE",
                ""
                "We can't ship to your address right now.
                Please contact customer service for
                alternatives.", false);
        case NetworkException ne ->
            new OrderResult(null,
                "TEMPORARY_ERROR",
                ""
                "We're experiencing technical difficulties.
                Please try again in a few minutes.", false);
        case SecurityException se -> new OrderResult(null,
            "SECURITY_CHECK_FAILED",
            ""

```



```

        Additional verification required.
        Please contact customer service.""", false);
    default -> ❷
        new OrderResult(null, "SYSTEM_ERROR",
            ""
            Something went wrong on our end. \
            Please try again or contact support.""", false);
    };
}
}

```

Let's examine the key points that make this exception-handling pattern effective:

- ❶ All three operations must succeed for order processing to complete; if any fails, the entire operation fails with a `FailedException`.
- ❷ Extracts the original exception from the `FailedException` wrapper to determine the specific failure type.
- ❸ Pattern matching maps each business exception type to a specific customer-friendly error response.
- ❹ The default case handles unexpected exceptions with a generic system error message.

The beauty of this approach lies in the `switch` expression that evaluates the cause of the failure. Each case addresses a specific business scenario with appropriate customer communication. Notice how we use `allSuccessfulOrThrow()` because order processing is an all-or-nothing operation—if any step fails, the entire order should fail, but with specific guidance about what went wrong.

Let's integrate the individual operations that make up our order-processing system so we can execute them. While these are merely placeholder methods, we might have similar functionality in a real application:

```

private String processPayment(String customerId, double amount)
    throws PaymentDeclinedException, NetworkException,
        SecurityException, InterruptedException {
    Thread.sleep(Duration.ofMillis(200));
    // Simulate different payment scenarios
    if (amount > 5000.0) {
        throw new SecurityException("High-value transaction requires" +
            "additional verification");
    }
    if (customerId.contains("declined")) {
        throw new PaymentDeclinedException("Insufficient funds");
    }
    if (customerId.contains("network")) {
        throw new NetworkException("Payment gateway timeout");
    }
}

```

```

        return "Payment processed: $" + amount;
    }

    private String checkAndReserveInventory(String productId)
        throws InsufficientInventoryException, NetworkException,
            InterruptedException {
        Thread.sleep(Duration.ofMillis(150));
        if (productId.contains("outofstock")) {
            throw new InsufficientInventoryException(
                "Only 0 items available, requested 1");
        }
        if (productId.contains("network")) {
            throw new NetworkException("Inventory service unavailable");
        }
        return "Reserved inventory for " + productId;
    }

    private String calculateShipping(String customerId, String productId)
        throws ShippingNotAvailableException, NetworkException,
            InterruptedException {
        Thread.sleep(Duration.ofMillis(100));
        if (customerId.contains("remote")) {
            throw new ShippingNotAvailableException(
                "No shipping available to remote location");
        }
        if (productId.contains("hazardous")) {
            throw new ShippingNotAvailableException(
                "Cannot ship hazardous materials to this address");
        }
        if (customerId.contains("network")) {
            throw new NetworkException("Shipping service unavailable");
        }
        return "Shipping calculated: $12.99";
    }

    private String generateOrderId() {
        return "ORD-" + System.currentTimeMillis();
    }
}

```

Each method simulates real-world scenarios that ecommerce developers face daily. Payment processing may fail due to insufficient funds or security holds. Inventory checks may reveal out-of-stock situations. Shipping calculations may uncover geographical or regulatory restrictions.

We define custom exception types that represent these specific business scenarios:

```

public static class PaymentDeclinedException extends Exception {
    public PaymentDeclinedException(String message) {
        super(message);
    }
}

```

```

public static class InsufficientInventoryException extends Exception {
    public InsufficientInventoryException(String message) {
        super(message);
    }
}

public static class ShippingNotAvailableException extends Exception {
    public ShippingNotAvailableException(String message) {
        super(message);
    }
}

public static class NetworkException extends Exception {
    public NetworkException(String message) {
        super(message);
    }
}

```

TIP

It's a best practice to create domain-specific exception types that represent business scenarios rather than technical failures. This makes your error handling more meaningful and your code more maintainable.

Now let's have the main method test it out:

```

void main() {
    var demo = new OrderProcessingService();
    var result1 = demo.processOrder("user123", "outofstock", 3500);
    System.out.println("Out of stock scenario: " + result1);
    var result2 = demo.processOrder("declined_user", "laptop", 1200);
    System.out.println("Payment declined scenario: " + result2);
    var result3 = demo.processOrder("remote_customer", "book", 25);
    System.out.println("Shipping unavailable scenario: " + result3);
    var result4 = demo.processOrder("user456", "laptop", 1200);
    System.out.println("Successful order: " + result4);
}

```

When we run this example, we will see how different failure scenarios produce specific, customer-friendly error messages. The

`FailedException` gives us access to the underlying cause, and pattern matching allows us to map each business exception to the appropriate customer communication.

Strategic exception propagation

Sometimes, the best strategy is to allow exceptions to propagate to higher levels, where they can be addressed with a broader context. This ap-

proach is especially useful when building layered applications that involve different responsibilities at various levels.

Consider a critical data-processing service where any failure should trigger system-wide alerts and fallback procedures:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.open;

public class ExceptionPropagationExample {

    // Let exceptions propagate for centralized handling
    public List<String> fetchCriticalData(List<String> sources)
        throws StructuredTaskScope.FailedException,
               InterruptedException { ❶

        try (var scope = open(StructuredTaskScope.Joiner.
            <String>allSuccessfulOrThrow())) {

            var tasks = sources.stream()
                .map(source -> scope.fork(()
                    -> fetchFromSource(source)))
                .toList();

            // If any source fails, let FailedException propagate
            var results = scope.join();

            return results.map(StructuredTaskScope.Subtask::get)
                .toList();
        }
        // No catch block - let FailedException propagate ❷
    }

    // Higher-level method with centralized exception handling
    public void processDataWithCentralizedHandling() {
        try {
            var sources = List.of("source1", "source2");
            var data = fetchCriticalData(sources);
            log("Successfully fetched data: " + data);
        } catch (StructuredTaskScope.FailedException e) { ❸
            // Centralized logging and error handling
            log("Critical data fetch failed: " +
                e.getCause().getMessage());

            // Could trigger alerts, fallback procedures, etc.
            handleCriticalSystemFailure(e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log("Operation was interrupted");
        }
    }
}
```

```

private String fetchFromSource(String source)
    throws InterruptedException {
    // Stub method
    return "Data from " + source;
}

private void handleCriticalSystemFailure(FailedException e) {
    Throwable cause = e.getCause();
    // Stub methods, feel free to add your own implementations
}
}

```

Here's how the exception propagation strategy works in this example:

- ❶ The method signature explicitly declares that it throws both `FailedException` and `InterruptedException`, signaling to callers that they must handle these exceptions.
- ❷ Deliberately omitting a `catch` block allows exceptions to propagate up the call stack to where they can be handled with more context.
- ❸ The higher-level method provides centralized exception handling that can implement comprehensive error responses like logging, alerting, and fallback procedures.

By allowing `FailedException` to propagate from `fetchCriticalData()`, we enable the calling method to implement comprehensive error handling that might include logging, alerting, and triggering fallback systems.

Best Practice: Use exception propagation when lower-level methods don't have sufficient context to handle failures appropriately. Let exceptions bubble up to components that can make informed decisions about recovery strategies.

NOTE

When propagating checked exceptions, ensure that intermediate layers properly declare thrown exceptions in their method signatures to maintain compile-time safety.

Handling exceptions within subtasks

For cases where specific exceptions should trigger default results rather than failing the entire operation, it's often better to handle exceptions within the subtask itself. This pattern is particularly useful for optional data sources where partial failure shouldn't prevent the entire operation from succeeding.

Imagine a dashboard service that gathers data from multiple optional services. If some services are unavailable, we still want to display the dashboard with whatever data we can gather:

```
import module java.base;
import static ca.bazlur.mcj.chap4.Uutils.log;
import static java.util.concurrent.StructuredTaskScope.*;

public class SubtaskExceptionHandler {
    public record ServiceResponse(String service, String data,
                                   boolean successful) {
    }
    public List<ServiceResponse> gatherOptionalData(List<String> services)
        throws InterruptedException {
        try (var scope
            = open(Joiner.<ServiceResponse>allSuccessfulOrThrow())) {
            var tasks = services.stream()
                .map(service -> scope.fork(() ->
                    fetchWithDefaults(service))) ❷
                .toList();

            var results = scope.join();
            return results.map(Subtask::get)
                .toList();

        } catch (FailedException e) { ❸
            // This should rarely happen since we handle
            // exceptions in subtasks
            log("Unexpected failure: " + e.getCause().getMessage());
            throw new RuntimeException("System error", e);
        }
    }

    // Handle exceptions within the subtask to provide defaults
    private ServiceResponse fetchWithDefaults(String service) { ❹
        try {
            String data = fetchServiceData(service);
            return new ServiceResponse(service, data, true);
        } catch (IOException e) {
            log("Network error for " + service + ": " + e.getMessage());
            return new ServiceResponse(service, "Default data", false);
        } catch (TimeoutException e) {
            log("Timeout for " + service + ": " + e.getMessage());
            return new ServiceResponse(service, "Cached data", false);
        } catch (Exception e) {
            log("Unexpected error for " + service + ": " +
                e.getMessage());
            return new ServiceResponse(service, "Error", false);
        }
    }

    private String fetchServiceData(String service)
        throws IOException, TimeoutException, InterruptedException {
```

```

        Thread.sleep(Duration.ofMillis(100));
        return "Data from " + service;
    }
}

```

Let's examine how this exception-handling strategy enables graceful degradation:

- ❶ Using `allSuccessfulOrThrow()` with subtasks that handle their own exceptions means this will rarely throw `FailedException`.
- ❷ Each service is fetched in a separate subtask that internally handles exceptions and provides fallback responses.
- ❸ The `catch` block handles truly unexpected failures, since individual service failures are handled within subtasks.
- ❹ Exception handling within the subtask converts failures into successful responses with default data, preventing individual service failures from cascading.

By handling exceptions within `fetchWithDefaults()`, we ensure that network failures or timeouts for individual services don't prevent the entire dashboard from loading. Each service attempt either succeeds with real data or fails gracefully with default or cached data.

Best Practice: Handle exceptions within subtasks when you want to implement graceful degradation. This pattern works well for optional operations where partial success is acceptable.

NOTE

When subtasks handle their own exceptions, the outer `FailedException` `catch` block should rarely execute. Consider logging when it does, as this may indicate an unexpected system issue.

Understanding how different joiners handle exceptions

The choice of joiner has a significant impact on how exceptions are handled. Let's compare the behavior of `allSuccessfulOrThrow()` and `awaitAll()` when dealing with failures.

The `allSuccessfulOrThrow()` joiner implements a fail-fast strategy where any single failure causes the entire operation to fail immediately:

```

private void demonstrateAllSuccessfulOrThrow() {
    try (var scope = open(StructuredTaskScope.Joiner.
        <String>allSuccessfulOrThrow())) {
        ❶
        scope.fork(() -> successfulTask("Task1"));
    }
}

```

```

        scope.fork(() -> failingTask("Task2")); ❷
        scope.fork(() -> successfulTask("Task3"));
        var results = scope.join(); ❸
        log("All tasks completed successfully");
    } catch (StructuredTaskScope.FailedException e) { ❹
        log("Failed due to: " + e.getCause().getMessage());
        log("Remaining tasks were canceled");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

```

Here are the key characteristics of the `allSuccessfulOrThrow()` approach:

- ❶ Creates a scope that requires all tasks to succeed or the entire operation fails.
- ❷ When this task throws an exception, it triggers the fail-fast behavior.
- ❸ The `join()` call will throw `FailedException` because `Task2` failed, and `Task3` may be canceled.
- ❹ Exception handling catches the failure and logs that the remaining tasks were canceled.

The `awaitAll()` joiner takes a different approach, allowing all tasks to complete regardless of individual failures:

```

private void demonstrateAwaitAll() {
    try (var scope = open(StructuredTaskScope.Joiner.
        <Void>awaitAll())) { ❶
        scope.fork(() -> successfulTask("Task1"));
        scope.fork(() -> failingTask("Task2")); ❷
        scope.fork(() -> successfulTask("Task3"));
        scope.join(); ❸
        log("All tasks were allowed to complete");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

```

Here are the key characteristics of the `awaitAll()` approach:

- ❶ The `awaitAll()` joiner waits for all tasks to complete, regardless of individual failures.
- ❷ `Task2` will still throw an exception, but it won't prevent other tasks from completing.
- ❸

The `join()` method never throws `FailedException` with `awaitAll()` —it waits for all tasks to finish, even if some fail.

The `allSuccessfulOrThrow()` joiner implements a fail-fast strategy: when `Task2` fails, it immediately cancels the remaining tasks and throws `FailedException`. In contrast, `awaitAll()` allows all tasks to complete regardless of individual failures, requiring us to handle exceptions within each subtask if we want to collect both successes and failures.

General exceptions

Unless otherwise specified, passing a `null` argument to a method in this class will cause a `NullPointerException` to be thrown. This consistent behavior helps catch programming errors early and makes the API more predictable.

Beyond the `null` pointer checks, the API throws specific exceptions for different error conditions, making it easier to handle problems appropriately.

Consider the following example:

```
import java.util.concurrent.Callable;
import java.util.concurrent.StructuredTaskScope;
import static ca.bazlur.mcj.chap4.Utils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class ExceptionBehaviorDemo {
    public void demonstrateCommonExceptions() {
        try {
            log("Testing null joiner...");
            try (var scope = open(null)) { ❶
                log("This should not be reached");
            }
        } catch (NullPointerException e) {
            log("NullPointerException caught for null joiner");
        }

        try {
            Callable<? extends String> nullCallable = null; ❷
            try (var scope = open(StructuredTaskScope.Joiner.
                <String>allSuccessfulOrThrow())) {
                scope.fork(nullCallable); ❸
            }
        } catch (NullPointerException e) {
            log("NullPointerException caught for null callable");
        }

        try {
```

```

        var scope = open(StructuredTaskScope.Joiner.
            <String>allSuccessfulOrThrow());
        scope.close(); ❷
        scope.fork(() -> "This should fail"); ❸
    } catch (IllegalStateException e) {
        log("IllegalStateException caught for closed scope operation");
    }
}

void main() {
    var demo = new ExceptionBehaviorDemo();
    demo.demonstrateCommonExceptions();
}
}

```

Let's examine each exception scenario:

- ❶ Attempting to open a scope with a `null` joiner immediately throws `NullPointerException`, preventing the scope from being created with an invalid configuration.
- ❷ Creates a `null` callable variable to demonstrate what happens when `null` tasks are submitted to the scope.
- ❸ Calling `fork()` with a `null` callable throws `NullPointerException`, ensuring that only valid tasks can be submitted for execution.
- ❹ Manually closes the scope before the `try-with-resources` block completes, putting the scope into a closed state.
- ❺ Attempting to fork a task on a closed scope throws `IllegalStateException`, preventing operations on invalid scope instances.

Let's run the preceding program:

```
java --enable-preview ExceptionBehaviorDemo.java
```

The output would be as follows:

```

23:14:21.314 main      : NullPointerException caught for null joiner
23:14:21.317 main      : NullPointerException caught for null callable
23:14:21.317 main      : IllegalStateException caught for closed scope ope

```

Configuration

While the default configuration of `StructuredTaskScope` works well for many scenarios, real-world applications often require more control over how concurrent tasks are executed. Configuration allows you to cus-

tomize thread creation, add monitoring capabilities, and implement timeout policies that align with your application's requirements.

Understanding Configuration

When we create a `StructuredTaskScope` using the simple `open()` or `open(Joiner)` methods, we get a sensible default configuration:

- `ThreadFactory`: Creates unnamed virtual threads
- Monitoring name: No name assigned (anonymous scope)
- Timeout: No timeout (tasks run until completion or cancellation)

```
// Uses default configuration
try (var scope = open(Joiner.allSuccessfulOrThrow())) { ❶
    // Virtual threads created with the default settings
    scope.fork(() -> doWork());
    scope.join();
}
```

- ❶ This default configuration creates anonymous virtual threads with no timeout.

For production applications, we will often need more sophisticated control. The Configuration API provides an overloaded `open` method that accepts a configuration function:

```
static <T, R> StructuredTaskScope<T, R>
    open(Joiner<? super T, ? extends R> joiner,
        Function<Configuration, Configuration> configFunction) {
    return StructuredTaskScopeImpl.open(joiner, configFunction);
}
```

The `Configuration` interface is a sealed interface that provides three configuration methods, each returning a new `Configuration` object with the specified modification:

```
sealed interface Configuration {
    Configuration withThreadFactory(ThreadFactory threadFactory);
    Configuration withName(String name);
    Configuration withTimeout(Duration timeout);
}
```

The configuration follows an immutable builder pattern—each method returns a new `Configuration` object rather than modifying the existing one. This allows you to chain multiple configuration calls together in a fluent style.

The key insight is that the configuration function transforms the default configuration by applying one or more modifications.

Here's the basic pattern:

```
try (var scope = open(Joiner.allSuccessfulOrThrow(),
    cf -> cf.withTimeout(Duration.ofSeconds(10)))) {
}
ThreadFactory factory = Thread.ofVirtual()
    .name("user-processor-", 0)
    .factory();
try (var scope = open(Joiner.allSuccessfulOrThrow(), cf -> cf
    .withThreadFactory(factory)
    .withTimeout(Duration.ofSeconds(30))
    .withName("my-scope"))) {
    // Continue with your code
}
```

Named threads

The most common customization is creating named threads to improve debugging and monitoring. Named threads make it much easier to identify which threads are doing which work in thread dumps and profiling tools.

Consider the following code snippets:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;

public class NamedThreadExample {
    public void processUserRequests(List<String> userIds) {
        ThreadFactory factory = Thread.ofVirtual()
            .name("user-processor-", 0)
            .factory();

        try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
            cf -> cf.withThreadFactory(factory))) {
            var tasks = userIds.stream()
                .map(userId -> scope.fork(() -> processUser(userId)))
                .toList();
            var results = scope.join()
                .map(Subtask::get)
                .toList();
            System.out.println("Processed users: " + results);
        } catch (FailedException | InterruptedException e) {
            System.out.println("Processing failed: " + e.getMessage());
        }
    }

    private String processUser(String userId)
```

```

        throws InterruptedException {
            System.out.println("Processing user " + userId +
                " on thread: " + Thread.currentThread().getName());
            Thread.sleep(Duration.ofMillis(100));
            return "User " + userId + " processed";
        }

        void main() {
            processUserRequests(List.of("user1", "user2", "user3"));
        }
    }
}

```

When we run this code, we'll see output like:

```

Processing user user2 on thread: user-processor-1
Processing user user3 on thread: user-processor-2
Processing user user1 on thread: user-processor-0

```

Timeout configuration

Timeouts are crucial for building resilient applications. They prevent operations from hanging indefinitely and allow your application to fail fast when services are unresponsive.

Consider the following code snippets:

```

import module java.base;
import static java.util.concurrent.StructuredTaskScope.*;

public class TimeoutExample {
    public List<String> fetchDataWithTimeout(List<String> sources)
        throws TimeoutException,
        FailedException,
        InterruptedException {
        Duration timeout = Duration.ofSeconds(5); ❶
        try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
            cf -> cf.withTimeout(timeout))) { ❷
            var tasks = sources.stream()
                .map(source -> scope.fork(() -> fetchFromSource(source)))
                .toList();
            // If timeout expires before join() completes,
            // TimeoutException is thrown
            return scope.join() ❸
                .map(Subtask::get)
                .toList();
        }
    }

    private String fetchFromSource(String source) throws InterruptedException {
        int delay = switch (source) {

```

```

        case "fast_source" -> 1000;
        case "slow_source" -> 3000;
        case "very_slow_source" -> 8000; ❹
        default -> 2000;
    };
    Thread.sleep(Duration.ofMillis(delay));
    return "Data from " + source;
}

void main() {
    var sources = List.of("fast_source", "slow_source", "very_slow_sour
    try {
        var results = fetchDataWithTimeout(sources);
        System.out.println("Success: " + results);
    } catch (TimeoutException e) { ❺
        System.out.println("Operation timed out after 5 seconds");
        System.out.println("Some sources were too slow to respond");
    } catch (FailedException e) {
        System.out.println("Task failed: " + e.getCause().getMessage());
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.out.println("Operation interrupted");
    }
}
}

```

Let's examine how the timeout mechanism works in this example:

- ❶ Sets a five-second timeout duration for the entire operation.
- ❷ Configures the scope with the timeout—the countdown begins when the scope opens.
- ❸ If any subtask hasn't completed within five seconds, `join()` throws `TimeoutException`.
- ❹ This source takes eight seconds, which exceeds our five-second timeout and will trigger the timeout.
- ❺ Catches and handles the timeout exception with user-friendly error messages.

If we run the preceding code, the output would be something like this:

```

Operation timed out after 5 seconds
Some sources were too slow to respond

```

NOTE

The timeout starts counting from the moment the scope is opened, not when `join()` is called. This means the timeout covers the entire lifecycle of the scope.

This demonstrates how timeouts provide fail-fast behavior when some operations take longer than expected, preventing your application from hanging indefinitely.

Combining configuration options

The most powerful aspect of configuration is the ability to combine various options to precisely construct the execution environment your application needs.

Consider the following code snippets:

```
import module java.base;

import static java.util.concurrent.StructuredTaskScope.*;

public class ComprehensiveConfigurationExample {

    public void main() {
        ThreadFactory factory = Thread.ofVirtual() ❶
            .name("api-client-", 0) ❷
            .factory(); ❸

        Duration timeout = Duration.ofSeconds(15); ❹

        try (var scope = open(Joiner.<String>allSuccessfulOrThrow(),
            cf -> cf.withThreadFactory(factory) ❺
                .withTimeout(timeout) ❻
                .withName("api-integration-scope"))) { ❼

            System.out.println("Starting comprehensive API integration");
            System.out.println("Timeout: " + timeout.getSeconds() + " seconds");

            // Fork multiple API calls
            var userTask = scope.fork(this::callUserAPI);
            var profileTask = scope.fork(this::callProfileAPI);
            var preferencesTask = scope.fork(this::callPreferencesAPI);

            var results = scope.join()
                .map(Subtask::get)
                .toList();

            System.out.println("All API calls completed: " + results);

        } catch (StructuredTaskScope.TimeoutException e) {
            System.out.println("API integration timed out - some services too slow");
        } catch (StructuredTaskScope.FailedException e) {
            System.out.println("API integration failed: " + e.getCause().getMessage());
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```

        System.out.println("API integration interrupted");
    }
}

private String callUserAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(3));
    IO.println(Thread.currentThread().getName()
        + " : " + "User data retrieved");
    return "User data";
}

private String callProfileAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(2));
    IO.println(Thread.currentThread().getName()
        + " : " + "Profile data retrieved");
    return "Profile data";
}

private String callPreferencesAPI() throws InterruptedException {
    Thread.sleep(Duration.ofSeconds(4));
    IO.println(Thread.currentThread().getName()
        + " : " + "Preferences data retrieved");
    return "Preferences data";
}
}

```

Let's examine this configuration step by step:

- ❶ Creates a builder for virtual threads, which are lightweight and ideal for I/O-bound operations like API calls
- ❷ Sets up a naming pattern where threads will be named `api-client-0`, `api-client-1`, `api-client-2`, etc., making them easily identifiable in debugging tools
- ❸ Converts the configured thread builder into a `ThreadFactory` that the scope can use to create threads
- ❹ Sets a 15-second timeout to ensure the operation fails fast if any API becomes unresponsive
- ❺ Configures the scope to use our custom `ThreadFactory` for creating worker threads
- ❻ Applies the timeout to the scope, starting the countdown when the scope is opened
- ❼ Assigns a descriptive name to the scope for monitoring and management purposes, making it easier to identify in production logs and monitoring tools

Custom Joiners

While the built-in joiners like `allSuccessfulOrThrow()` and `awaitAll()` offer a solid foundation for task coordination and result management, structured concurrency doesn't stop there. It allows developers to create custom joiners to meet specific application needs. This flexibility unlocks a whole new realm of possibilities for orchestrating concurrent workflows with precise control over how subtasks are managed and results are collected.

Custom joiners can be created by implementing the `StructuredTaskScope.Joiner<T, R>` interface. This interface provides methods to handle subtask completion events and determine when the scope should complete and what result it should produce.

Every joiner implements the `StructuredTaskScope.Joiner<T, R>` interface, where `T` is the type of values produced by subtasks and `R` is the type of the final result. The interface defines three key methods:

`onFork(Subtask<? extends T> subtask)`

Called each time a subtask is forked, before the thread is created to run it

`onComplete(Subtask<? extends T> subtask)`

Called each time a subtask completes, allowing you to process the result and decide whether to continue or complete early

`result()`

Called when the scope is ready to produce its final result

Let's explore several practical examples that demonstrate the power and flexibility of custom joiners.

Collecting all results and exceptions

Let's consider a scenario where we want to gather the results of all subtasks, whether they succeed or fail, and collect any exceptions that occur. This pattern is helpful for batch processing, where we want maximum information about what succeeded and what failed:

```
import module java.base;

public class CollectingJoiner<T>
    implements StructuredTaskScope.Joiner<T, CollectingJoiner.Result<T>
    private final Queue<T> results = new ConcurrentLinkedQueue<>();
    private final Queue<Throwable> exceptions = new ConcurrentLinkedQueue<>()

    @Override
    public Result<T> result() {
```

```

        return new Result<>(
            results.stream().toList(),
            exceptions.stream().toList()
        );
    }

    @Override
    public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
        switch (subtask.state()) {
            case SUCCESS -> results.add(subtask.get()); ❶
            case FAILED -> exceptions.add(subtask.exception()); ❷
            case UNAVAILABLE -> { ❸
                // Task was canceled, treat as failure
                exceptions.add(new RuntimeException("Task was canceled"));
            }
        }
        return false; ❹
    }

    public record Result<T>(List<T> successes, List<Throwable> failures) {
        public boolean hasFailures() {
            return !failures.isEmpty();
        }
        public int totalTasks() {
            return successes.size() + failures.size();
        }
    }
}

```

Let's examine how this joiner handles different subtask outcomes:

- ❶ When a subtask completes successfully, we add its result to the results queue for later retrieval.
- ❷ When a subtask fails with an exception, we collect the exception for analysis rather than immediately failing the entire operation.
- ❸ Tasks in the `UNAVAILABLE` state were canceled before completion. We treat cancellation as a failure and create a descriptive exception.
- ❹ Returning `false` allows all remaining subtasks to continue running until they naturally complete.

Now let's use this joiner to build a service that fetches headlines from various online sources, collecting as many as possible, even if some sources fail:

```

import module java.base;
import static ca.bazlur.mcj.chap4.Uutils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class NewsAggregator {

```

```

public CollectingJoiner.Result<String> fetchAllHeadlines()
    throws InterruptedException {
    var newsSources = List.of(
        "TechCrunch",
        "InfoWorld",
        "InfoQ",
        "FailingSource"
    );

    try (var scope = open(new CollectingJoiner<String>())) {
        for (String source : newsSources) {
            scope.fork(() -> fetchHeadlines(source)); ❶
        }
        var result = scope.join(); ❷
        log("Successfully fetched from " + result.successes().size() +
            " sources");
        log("Failed to fetch from " + result.failures().size() +
            " sources");
        return result; ❸
    }
}

private String fetchHeadlines(String source) throws InterruptedException
    log("Fetching headlines from " + source);
    Thread.sleep(Duration.ofMillis(200 + new Random().nextInt(300)));
    if (source.equals("FailingSource")) { ❹
        throw new RuntimeException("Network timeout for " + source);
    }
    return "Headlines from " + source + ": Breaking news, Tech updates";
}

void main() {
    var aggregator = new NewsAggregator();
    try {
        var result = aggregator.fetchAllHeadlines();
        log("\n=== Results ===");
        result.successes() ❺
            .forEach(headline -> log(headline));
        if (result.hasFailures()) { ❻
            log("\n=== Failures ===");
            result.failures()
                .forEach(error ->
                    log(error.getMessage()));
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        log("Operation interrupted");
    }
}
}

```

This example demonstrates the power of collecting both successes and failures:

- ❶ Launches a subtask for each news source simultaneously, allowing them to run in parallel.
- ❷ The `CollectingJoiner` waits for all subtasks to complete, collecting both successful headlines and any failures that occur.
- ❸ Returns a result that contains both the successful data and information about what failed.
- ❹ One source is designed to fail, demonstrating how the joiner handles mixed outcomes.
- ❺ Displays all successfully fetched headlines, giving users the maximum available information.
- ❻ Reports on failures without letting them prevent the display of successful results.

When we run this example, we'll see output as follows:

```
00:53:44.291 VThread[#34]: Fetching headlines from TechCrunch
00:53:44.291 VThread[#42]: Fetching headlines from FailingSource
00:53:44.291 VThread[#39]: Fetching headlines from InfoQ
00:53:44.291 VThread[#36]: Fetching headlines from InfoWorld
00:53:44.773 main      : Successfully fetched from 3 sources
00:53:44.773 main      : Failed to fetch from 1 sources
00:53:44.773 main      :
=== Results ===
01:43:17.431 main      : Headlines from InfoWorld: Breaking news, Tech up
01:43:17.431 main      : Headlines from TechCrunch: Breaking news, Tech u
01:43:17.431 main      : Headlines from InfoQ: Breaking news, Tech update
01:43:17.431 main      :
=== Failures ===
01:43:17.431 main      : Network timeout for FailingSource
```

Quorum-based completion

Let's implement a joiner inspired by distributed systems that completes when a quorum of subtasks succeeds. This is useful for scenarios where you need a certain number of confirmations before proceeding:

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.atomic.AtomicInteger;

public class QuorumJoiner<T> implements StructuredTaskScope.Joiner<T, Boolean> {
    private final int requiredSuccesses;
    private final AtomicInteger successCount = new AtomicInteger(0); ❶
    private final AtomicInteger totalCount = new AtomicInteger(0); ❷
    private volatile boolean quorumReached = false; ❸

    public QuorumJoiner(int requiredSuccesses) {
        this.requiredSuccesses = requiredSuccesses;
    }
}
```

```

@Override
public Boolean result() {
    return quorumReached; ❹
}

@Override
public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
    totalCount.incrementAndGet(); ❺
    return false; // Allow all tasks to proceed ❻
}

@Override
public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    if (subtask.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
        int currentSuccess = successCount.incrementAndGet(); ❼
        if (currentSuccess >= requiredSuccesses) { ❽
            quorumReached = true;
            return true; ❾
        }
    }
    return false; ❿
}

public int getSuccessCount() {
    return successCount.get();
}

public int getTotalCount() {
    return totalCount.get();
}
}

```

Let's examine how this quorum-based joiner manages consensus:

- ❶ Tracks the number of subtasks that have completed successfully using an atomic counter for thread safety.
- ❷ Keeps track of how many subtasks have been forked, useful for calculating completion percentages or remaining tasks.
- ❸ A volatile boolean that indicates whether the required quorum has been reached, ensuring visibility across threads.
- ❹ Returns whether the quorum was successfully achieved by the time the scope was completed.
- ❺ Increments the total count each time a subtask is forked, giving us visibility into the total number of tasks.
- ❻ Returns `false` to allow all subtasks to be created; we don't want to prevent task creation at fork time.
- ❼ When a subtask succeeds, atomically increments the success counter and captures the new count.

- ⑧ Tests whether we've reached the required number of successes for our quorum.
- ⑨ Once we have enough successes, sets the flag and returns `true` to cancel remaining tasks and complete the scope.
- ⑩ If we don't have enough successes yet, returns `false` to let other tasks continue running.

Here's how to use the quorum joiner for a distributed database write operation:

```
import java.time.Duration;
import java.util.List;
import java.util.Random;
import static ca.bazlur.mcj.chap4.Utills.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class DistributedDatabase {
    private final List<String> nodes = List.of(
        "node-1", "node-2", "node-3", "node-4", "node-5" ①
    );
    private final int quorumSize = 3; // Need 3 out of 5 nodes ②

    public boolean writeData(String key, String value)
        throws InterruptedException {
        log("Writing data to distributed database:\n " + key + "=" + value);
        try (var scope = open(new QuorumJoiner<Boolean>(quorumSize))) { ③
            for (String node : nodes) {
                scope.fork(() -> writeToNode(node, key, value)); ④
            }
            boolean success = scope.join(); ⑤
            log("Write operation " + (success ? "succeeded" : "failed"));
            log("Required quorum: " + quorumSize + " nodes");
            return success;
        }
    }

    private Boolean writeToNode(String node, String key, String value)
        throws InterruptedException {
        log("Writing to " + node);
        Thread.sleep(Duration.ofMillis(100 + new Random().nextInt(200))); ⑥
        // Simulate occasional node failures
        if (new Random().nextDouble() < 0.2) { ⑦
            log(node + " write failed");
            throw new RuntimeException("Write failed on " + node);
        }
        log(node + " write succeeded");
        return true;
    }

    void main() {
        var database = new DistributedDatabase();
    }
}
```

```

    try {
        boolean result = database.writeData("user:123", "John Doe"); ❸
        log("\nFinal result: " + (result ? "SUCCESS" : "FAILURE"));
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        log("Database operation interrupted");
    }
}
}
}

```

Let's examine how this works:

- ❶ Defines a cluster of five database nodes. In a real system, these would be different physical servers or containers.
- ❷ Sets quorum size to three out of five nodes. This provides fault tolerance while maintaining availability; the system can survive up to two node failures.
- ❸ Creates a `StructuredTaskScope` with a custom `QuorumJoiner` that completes when the required number of successful operations is reached, rather than waiting for all tasks.
- ❹ Forks a separate virtual thread for each node write operation. All writes execute concurrently, maximizing parallelism and reducing total latency.
- ❺ Blocks until quorum is achieved or determined impossible. The `QuorumJoiner` returns `true` if enough writes succeed, `false` if too many fail to reach quorum.
- ❻ Simulates realistic network latency with random delays between 100 and 300ms, representing typical database write times across a network.
- ❼ Introduces a 20% failure rate to simulate real-world distributed system challenges like network partitions, disk failures, or node crashes.
- ❽ The high-level API abstracts away all the complexity of distributed coordination, presenting a simple synchronous interface to callers.

If we now run the preceding program, we will get the following output:

```

21:11:24.267 main          : Writing data to distributed database:
user:123=John Doe
01:18:38.141 VThread[#34]: Writing to node-1
01:18:38.142 VThread[#36]: Writing to node-2
01:18:38.141 VThread[#44]: Writing to node-5
01:18:38.142 VThread[#42]: Writing to node-4
01:18:38.142 VThread[#39]: Writing to node-3
01:18:38.252 VThread[#42]: node-4 write succeeded
01:18:38.285 VThread[#34]: node-1 write succeeded
01:18:38.364 VThread[#39]: node-3 write failed
01:18:38.376 VThread[#36]: node-2 write succeeded

```

```

01:18:38.378 main      : Write operation succeeded
01:18:38.378 main      : Required quorum: 3 nodes
01:18:38.379 main      :
Final result: SUCCESS

```

Adaptive completion

Sometimes we may want a completion logic that adapts based on the results we're seeing. Let's create a joiner that completes early if it sees too many failures:

```

import module java.base;

public class AdaptiveJoiner<T>
    implements StructuredTaskScope.Joiner<T, CollectingJoiner.Result<T>> {
    private final int minSampleSize;
    private final double maxFailureRate;
    private final Queue<T> successes = new ConcurrentLinkedQueue<>();
    private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

    public AdaptiveJoiner(double maxFailureRate, int minSampleSize) {
        this.maxFailureRate = maxFailureRate;
        this.minSampleSize = minSampleSize;
    }

    @Override
    public CollectingJoiner.Result<T> result() {
        return new CollectingJoiner.Result<>(
            successes.stream().toList(),
            failures.stream().toList()
        );
    }

    @Override
    public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
        switch (subtask.state()) {
            case SUCCESS -> successes.add(subtask.get()); ❶
            case FAILED -> failures.add(subtask.exception()); ❶
            case UNAVAILABLE -> failures.add(
                new RuntimeException("Task canceled"));
        }
        int total = successes.size() + failures.size();
        // Only check failure rate after minimum sample size
        if (total >= minSampleSize) { ❷
            double failureRate = (double) failures.size() / total;
            return failureRate > maxFailureRate; ❸
        }
        return false; // Continue processing ❹
    }
}

```


Now let's explain how it works:

- ❶ Collects both successes and failures as they occur.
- ❷ Only evaluates the failure rate after we have enough data points to make a meaningful decision.
- ❸ If the failure rate exceeds our threshold, complete early to avoid wasting resources on a likely failing operation.
- ❹ If the failure rate is acceptable or we don't have enough data yet, continue processing.

Let's now use it in an example:

```
import module java.base;
import static ca.bazlur.mcj.chap4.rewrite.Uutils.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class WebCrawlerWithCircuitBreaker {
    private final List<String> urls = List.of(
        "https://api.service1.com/data", ❶
        "https://api.service2.com/data",
        "https://api.service3.com/data",
        "https://api.service4.com/data",
        "https://api.service5.com/data"
    );
    private final Random random = new Random();
    private double systemFailureRate = 0.1; // Start with 10% failure rate

    public CollectingJoiner.Result<String> crawlWithCircuitBreaker()
        throws InterruptedException {
        log("Starting web crawl with circuit breaker protection");
        log("Max failure rate: 30%, Min sample size: 5");
        // Create adaptive joiner that stops if >30% fail after 5 samples
        try (var scope = open(new AdaptiveJoiner<String>(0.30, 5))) { ❷
            for (String url : urls) {
                scope.fork(() -> fetchUrl(url)); ❸
            }
            var result = scope.join(); ❹
            log("\n=== FINAL RESULTS ===");
            log("Successful fetches: " + result.successes().size());
            log("Failed fetches: " + result.failures().size());
            log("Total processed: " + (result.successes().size()
                + result.failures().size()));
            log("Remaining URLs (not processed): " +
                (urls.size() - result.successes().size() - result.failures().size()));
            return result;
        }
    }

    private String fetchUrl(String url) throws InterruptedException {
        log("Fetching: " + url);
        // Simulate network delay
    }
}
```

```

        Thread.sleep(Duration.ofMillis(200 + random.nextInt(300))); ❸
        // Simulate system degradation over time
        if (random.nextDouble() < systemFailureRate) { ❹
            systemFailureRate += 0.05; // Failures increase over time
            throw new RuntimeException("Network timeout for " + url);
        }
        log("✓ Successfully fetched: " + url);
        return "Data from " + url;
    }

    public void demonstrateScenarios() throws InterruptedException {
        log("=== SCENARIO 1: Normal Operation ===");
        systemFailureRate = 0.1; // Low failure rate
        crawlWithCircuitBreaker();
        Thread.sleep(1000);
        log("\n=== SCENARIO 2: System Under Stress ===");
        systemFailureRate = 0.4; // High initial failure rate
        crawlWithCircuitBreaker();
        Thread.sleep(1000);
        log("\n=== SCENARIO 3: Cascading Failures ===");
        systemFailureRate = 0.6; // Very high failure rate
        crawlWithCircuitBreaker();
    }

    void main() {
        var crawler = new WebCrawlerWithCircuitBreaker();
        try {
            crawler.demonstrateScenarios();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log("Crawling interrupted");
        }
    }
}

```

Now let's see what's going on in the preceding code:

- ❶ Defines a list of API endpoints to crawl. In a real system, this might be thousands of URLs from a web crawler queue or API discovery service.
- ❷ Configures the adaptive joiner with failure tolerance settings. A 30% max failure rate with five minimum samples provides a good balance between responsiveness and stability.
- ❸ Forks concurrent HTTP requests for each URL. All requests execute in parallel using virtual threads, maximizing throughput while the system is healthy.
- ❹ Blocks until completion or circuit breaker activation. The adaptive joiner will return early if too many failures occur, saving time and resources.
- ❺

Simulates realistic network conditions with variable latency. Real HTTP requests have unpredictable response times due to network conditions and server load.

- ⑥ Models system degradation where failures compound over time. This simulates real-world scenarios like cascading failures, memory leaks, or infrastructure problems that worsen under load.

Now, if we run this and get the output, it will be as follows:

```
01:55:50.225 main      : === SCENARIO 1: Normal Operation ===
01:55:50.226 main      : Starting web crawl with circuit breaker protecti
01:55:50.226 main      : Max failure rate: 30%, Min sample size: 5
01:55:50.269 VThread[#42]: Fetching: https://api.service4.com/data
01:55:50.270 VThread[#45]: Fetching: https://api.service5.com/data
... [3 more fetch operations] ...
01:55:50.473 VThread[#39]: ✓ Successfully fetched: https://api.service3.com
... [3 more successful fetches] ...
01:55:50.753 main      : === FINAL RESULTS ===
01:55:50.753 main      : Successful fetches: 4, Failed fetches: 1

01:55:51.759 main      : === SCENARIO 2: System Under Stress ===
... [initialization and fetch operations] ...
01:55:52.182 main      : Successful fetches: 3, Failed fetches: 2

01:55:53.188 main      : === SCENARIO 3: Cascading Failures ===
... [initialization and fetch operations] ...
01:55:53.653 main      : Successful fetches: 0, Failed fetches: 5
=== FINAL RESULTS ===
01:55:53.653 main      : Successful fetches: 0
01:55:53.653 main      : Failed fetches: 5
01:55:53.653 main      : Total processed: 5
01:55:53.653 main      : Remaining URLs (not processed): 0
```

Rate-limited joiner

Sometimes we need to control the rate at which subtasks are executed to avoid overwhelming external services. Let's create a joiner that implements rate limiting using `Semaphore`:

```
import module java.base;

public class RateLimitedJoiner<T>
    implements StructuredTaskScope.Joiner<T, List<T>> {
    private final Semaphore semaphore; ①
    private final Queue<T> results = new ConcurrentLinkedQueue<>();
    private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

    public RateLimitedJoiner(int maxConcurrentTasks) {
        this.semaphore = new Semaphore(maxConcurrentTasks); ②
    }
}
```

```

@Override
public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
    try {
        semaphore.acquire(); ❸
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    return false; ❹
}

@Override
public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
    switch (subtask.state()) {
        case SUCCESS -> results.add(subtask.get()); ❺
        case FAILED -> failures.add(subtask.exception()); ❻
        case UNAVAILABLE -> failures.add(new RuntimeException("Task canceled"));
    }
    semaphore.release(); ❼
    return false; ❽
}

@Override
public List<T> result() {
    return results.stream().toList();
}
}

```

Let's examine how this semaphore-based rate limiting works:

- ❶ Uses a `Semaphore` to control the number of concurrent executions allowed.
- ❷ Creates the `Semaphore` with the maximum number of concurrent tasks allowed.
- ❸ Acquires a permit before the task starts execution. If no permits are available, this call blocks until one becomes available, effectively controlling the rate of task execution.
- ❹ Returns `false` to allow the subtask to be created and start execution (now that we have a permit).
- ❺ Collects successful results when tasks are completed successfully.
- ❻ Collects failures to maintain visibility into what went wrong.
- ❼ Always releases the semaphore permit when a task completes, regardless of success or failure, making it available for other waiting tasks.
- ❽ Allows all tasks to complete rather than stopping early.

Let's demonstrate the rate-limited joiner with a batch API-calling scenario:

```

import module java.base;
import static ca.bazlur.mcj.chap4.Utills.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class RateLimitedAPIService {
    public List<String> fetchDataWithRateLimit() throws InterruptedException {
        var endpoints = List.of(
            "api/users", "api/orders", "api/products",
            "api/analytics", "api/reports", "api/logs"
        );
        // Allow maximum 3 concurrent API calls
        var rateLimitedJoiner = new RateLimitedJoiner<String>(3);
        try (var scope = open(rateLimitedJoiner)) {
            for (String endpoint : endpoints) {
                scope.fork(() -> callAPI(endpoint)); ❶
            }
            var results = scope.join(); ❷
            log("Completed " + results.size() +
                " API calls with rate limiting");
            return results;
        }
    }

    private String callAPI(String endpoint) throws InterruptedException {
        log("Starting API call: " + endpoint );
        Thread.sleep(Duration.ofMillis(500 + new Random().nextInt(1000)));
        log("Completed API call: " + endpoint);
        return "Response from " + endpoint;
    }

    void main() {
        var service = new RateLimitedAPIService();
        try {
            var results = service.fetchDataWithRateLimit();
            log("All results: " + results);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log("Service interrupted");
        }
    }
}

```

Let's see how it works:

- ❶ Each `fork()` call will acquire a semaphore permit before the task starts executing, ensuring only three tasks run concurrently.
- ❷ Waits for all API calls to complete, with the semaphore automatically managing the concurrency.

When we run this example with six endpoints but a limit of three concurrent tasks, we'll see output as follows:

```

02:28:43.648 VThread[#38]: Starting API call: api/products
02:28:43.648 VThread[#36]: Starting API call: api/orders
02:28:43.648 VThread[#34]: Starting API call: api/users
02:28:44.266 VThread[#34]: Completed API call: api/users
02:28:44.268 VThread[#44]: Starting API call: api/analytics
02:28:44.499 VThread[#38]: Completed API call: api/products
02:28:44.501 VThread[#46]: Starting API call: api/reports
02:28:45.021 VThread[#36]: Completed API call: api/orders
02:28:45.022 VThread[#47]: Starting API call: api/logs
02:28:45.381 VThread[#44]: Completed API call: api/analytics
02:28:45.849 VThread[#47]: Completed API call: api/logs
02:28:45.974 VThread[#46]: Completed API call: api/reports
02:28:45.975 main      : Completed 6 API calls with rate limiting
02:28:45.975 main      : All results: [Response from api/users, Response
api/products, Response from api/orders, Response from api/analytics, Respon
from api/logs, Response from api/reports]

```

Conditional joiners

The `onFork` method can also be used to implement conditional execution based on runtime state. For example, a generic joiner that accepts any condition for deciding when to stop new tasks.

Let's implement this idea:

```

import module java.base;

public class ConditionalJoiner<T>
    implements StructuredTaskScope.Joiner<T, List<T>> {
    private final Supplier<Boolean> shouldContinue; ❶
    private final Queue<T> results = new ConcurrentLinkedQueue<>();
    private final Queue<Throwable> failures = new ConcurrentLinkedQueue<>();

    public ConditionalJoiner(Supplier<Boolean> shouldContinue) {
        this.shouldContinue = shouldContinue; ❷
    }

    @Override
    public boolean onFork(StructuredTaskScope.Subtask<? extends T> subtask) {
        if (!shouldContinue.get()) { ❸
            System.out.println("Condition failed, stopping new tasks");
            return true; // Cancel scope to prevent new work ❹
        }
        return false; // Condition satisfied, allow task ❺
    }

    @Override
    public boolean onComplete(StructuredTaskScope.Subtask<? extends T> subtask) {
        switch (subtask.state()) {
            case SUCCESS -> results.add(subtask.get()); ❻
            case FAILED -> failures.add(subtask.exception()); ❼
        }
    }
}

```

```

        case UNAVAILABLE -> failures.add(new RuntimeException("Task canceled"))
    }
    return false; // Continue processing existing tasks ⑧
}

@Override
public List<T> result() {
    return results.stream().toList();
}

public List<Throwable> getFailures() {
    return failures.stream().toList();
}
}

```

Let's examine how this joiner handles different subtask outcomes:

- ❶ Accepts any condition as a `Supplier<Boolean>` that determines whether to continue processing
- ❷ Stores the condition supplier for evaluation during task forking
- ❸ Checks the condition each time a new task wants to start
- ❹ If condition fails, stop the scope to prevent new tasks from starting
- ❺ When condition passes, allow tasks to proceed normally
- ❻ Collects successful results as usual
- ❼ Keeps track of failures for analysis
- ❽ Allows already-running tasks to complete

Now let's apply this to a simple example:

```

package ca.bazlur.modern.concurrency.c04;

import java.time.Duration;
import java.util.List;
import java.util.Random;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.atomic.AtomicInteger;

import static java.util.concurrent.StructuredTaskScope.open;

public class SystemHealthCheckDemo {

    void main() throws InterruptedException {
        var results = new SystemHealthCheckDemo()
            .processWithSystemHealthCheck();
        results.forEach(System.out::println);
    }

    public List<String> processWithSystemHealthCheck()

```

```

        throws InterruptedException {
var healthChecker = new SystemHealthChecker();
var joiner = new ConditionalJoiner<String>(healthChecker::isSystemHealthy);

try (var scope = open(joiner)) {
    for (int i = 0; i < 10; i++) {
        int id = i;
        scope.fork(() -> processTask("health-task-" + id));
    }
    return scope.join();
}

private String processTask(String taskName)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(200
        + new Random().nextInt(300)));

    if (taskName.contains("error-task")
        && new Random().nextDouble() < 0.4)
        throw new RuntimeException("Task failed: " + taskName);

    return "Completed: " + taskName;
}
}

```

In the preceding code, we check an external system's health and stop if dependencies become unavailable.

The simple `SystemHealthChecker` class looks as follows:

```

private static class SystemHealthChecker {
    private final AtomicInteger checkCount
        = new AtomicInteger(0);

    public boolean isSystemHealthy() {
        checkCount.incrementAndGet();
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        return checkCount.get() < 7;
    }
}

```

Custom joiners represent one of the most powerful aspects of structured concurrency in Java. They transform `StructuredTaskScope` from a simple parallel execution tool into a sophisticated coordination platform that can adapt to virtually any concurrent programming challenge.

The beauty of the joiner pattern lies in its simplicity and flexibility. With just three methods, `onFork`, `onComplete`, and `result`, we can implement arbitrarily complex coordination logic while maintaining the safety guarantees and clean structure that make structured concurrency so powerful.

The examples we've explored are just the beginning. The real power emerges when you start designing joiners that capture the unique coordination patterns in your own domain, creating reusable components that make concurrent programming not just safer, but genuinely elegant.

Memory Consistency Effects

`StructuredTaskScope` provides well-defined memory consistency guarantees that make it easier to reason about the visibility of shared data across threads.

Actions in the owner thread of a `StructuredTaskScope` prior to the forking of a subtask happen before any actions taken by that subtask, which in turn [happen before](#) the subtask result is retrieved. This *happens-before* relationship ensures that any data modifications made in the owner thread before calling `fork()` are visible to the subtask, and any changes made by the subtask are visible when retrieving its result.

MEMORY CONSISTENCY PROPERTIES

`StructuredTaskScope`'s memory consistency guarantees rest on Java's fundamental happens-before relationships, as defined in the [Java Language Specification \(Chapter 17\)](#). These rules ensure that memory operations across threads occur in a predictable, safe manner.

Java provides several built-in happens-before relationships that make concurrent programming safe:

- Actions within a single thread execute in program order.
- Synchronized blocks create ordering between threads through monitor operations.
- Volatile variables provide visibility guarantees without locking.
- Thread creation and joining establish clear memory boundaries.

The `java.util.concurrent` package builds upon these foundation rules to provide higher-level guarantees. Operations like submitting tasks to executors, placing items in concurrent collections, and retrieving `Future` results all establish happens-before relationships that ensure thread safety.

Let's see this guarantee in action with a realistic example. Let's imagine we are building a configuration service that needs to validate settings across multiple systems simultaneously:

```
import module java.base;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import static ca.bazlur.mcj.chap4.Utills.log;
import static java.util.concurrent.StructuredTaskScope.open;

public class MemoryConsistencyDemo {
    private String configuration = "default";
    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public void demonstrateMemoryConsistency() throws InterruptedException {
        // Owner thread sets up a shared state before forking
        configuration = "production-config"; ❶
        cache.put("database-url", "prod.example.com"); ❶
        cache.put("api-key", "secret-key-123"); ❶
        log("Owner thread prepared: " + configuration);
        try (var scope = open(StructuredTaskScope.Joiner.
            <String>allSuccessfulOrThrow())) {
            // Fork subtasks that read owner thread's data
            var configTask = scope.fork(() -> { ❷
                log("Subtask sees: " + configuration); ❸
                return "Config: " + configuration;
            });
            var cacheTask = scope.fork(() -> { ❷
                String url = cache.get("database-url"); ❸
                log("Subtask found URL: " + url);
                return "Connected to: " + url;
            });
            var results = scope.join() ❹
                .map(StructuredTaskScope.Subtask::get)
                .toList();
            log("Owner received: " + results); ❺
        }
    }
}
```

Let's trace through what happens:

- ❶ Before creating any subtasks, the owner thread prepares the shared state. It updates the configuration string and populates the cache with essential connection details.
- ❷ When we call `fork()`, something important happens under the hood. The structured concurrency framework establishes a happens-before relationship, ensuring that all the preparations we made in steps 1, 2, and 3 become visible to the subtasks.
- ❸

Here's where the magic happens. Without any additional synchronization on our part, both subtasks can reliably see the updated configuration and cache contents. The configuration variable shows `production-config` (not `default`), and the cache contains all the entries we added.

- ④ The `join()` operation creates another happens-before edge, this time from the subtasks back to the owner thread.
- ⑤ The owner thread now sees all the results from the subtasks, guaranteed to be up to date and consistent.

The happens-before guarantee works in both directions. When subtasks modify shared state, those changes become visible to the owner thread after the join operation completes.

Let's examine another snippet:

```
public void demonstrateSubtaskUpdates() throws InterruptedException {
    var results = new ConcurrentHashMap<String, String>(); ①
    var counter = new AtomicInteger(0); ①
    log("Initial count: " + counter.get());

    try (var scope = open(StructuredTaskScope.Joiner.
        <String>allSuccessfulOrThrow())) {
        var worker1 = scope.fork(() -> { ②
            results.put("task1", "completed"); ③
            int count = counter.incrementAndGet(); ④
            log("Worker1 incremented to: " + count);
            return "Worker1 done";
        });
        var worker2 = scope.fork(() -> { ②
            results.put("task2", "completed"); ③
            int count = counter.incrementAndGet(); ④
            log("Worker2 incremented to: " + count);
            return "Worker2 done";
        });
        scope.join(); ⑤
        // Owner thread sees all subtask updates
        log("Final count: " + counter.get()); ⑥
        log("Final results: " + results); ⑥
    }
}
```

This example shows how multiple subtasks can safely update shared state:

- ① We start by creating thread-safe containers, a `ConcurrentHashMap` for collecting results and an `AtomicInteger` for counting operations.
- ②

We launch two workers that will run simultaneously. They might execute in any order, but both will safely update our shared containers.

- ❸ Each worker adds an entry to the results map. Because we're using `ConcurrentHashMap`, these operations are thread-safe even when happening concurrently.
- ❹ The atomic counter ensures that increments from different threads don't interfere with each other, preventing the classic "lost update" problem.
- ❺ The `join()` operation ensures that all worker modifications are visible to the owner thread.
- ❻ After the join completes, the owner thread sees the final state of both the counter and the results map, with all updates from both workers properly reflected.

As we can see, `StructuredTaskScope` leverages these proven mechanisms to provide its guarantees. When we fork a subtask, it's similar to starting a thread, creating a happens-before edge that makes all prior owner thread actions visible to the subtask. When we join, it's like calling `Thread.join()`, establishing a happens-before relationship that makes all subtask actions visible to the owner thread.

The concurrent collections from `java.util.concurrent` extend these guarantees further, ensuring that operations like putting items into a `ConcurrentHashMap` or incrementing an `AtomicInteger` are safely visible across threads.

This automatic memory consistency is one of the key advantages of structured concurrency. We can focus on the business logic of our concurrent operations, confident that the memory consistency details are handled correctly. The fork and join operations provide clear synchronization points where we know all necessary memory barriers are in place. However, this doesn't mean we can ignore thread safety entirely. We still need to use appropriate concurrent collections and atomic operations when multiple subtasks modify shared state.

Nested Scopes

We've explored how to create custom joiners and manage concurrent tasks within a single scope, but the power of structured concurrency extends further. Just as we can nest code blocks within methods, we can nest `StructuredTaskScope` instances to create sophisticated, multilevel hierarchies of concurrent tasks.

Nested scopes are invaluable for managing complex workflows where tasks naturally break down into multiple levels of subtasks. Each nested

scope is self-contained, encapsulating its subtasks' lifecycle and error handling.

When a subtask creates its own nested `StructuredTaskScope`, a parent-child relationship is established, forming a tree structure. The parent scope becomes responsible for the child scope's lifecycle, including shutdown and completion of its subtasks.

This hierarchical structure ([Figure 4-3](#)) propagates errors and cancellations upward. If a subtask in a nested scope fails, the error is first handled by its immediate parent scope, then propagated up the chain until it's handled or reaches the top-level scope.

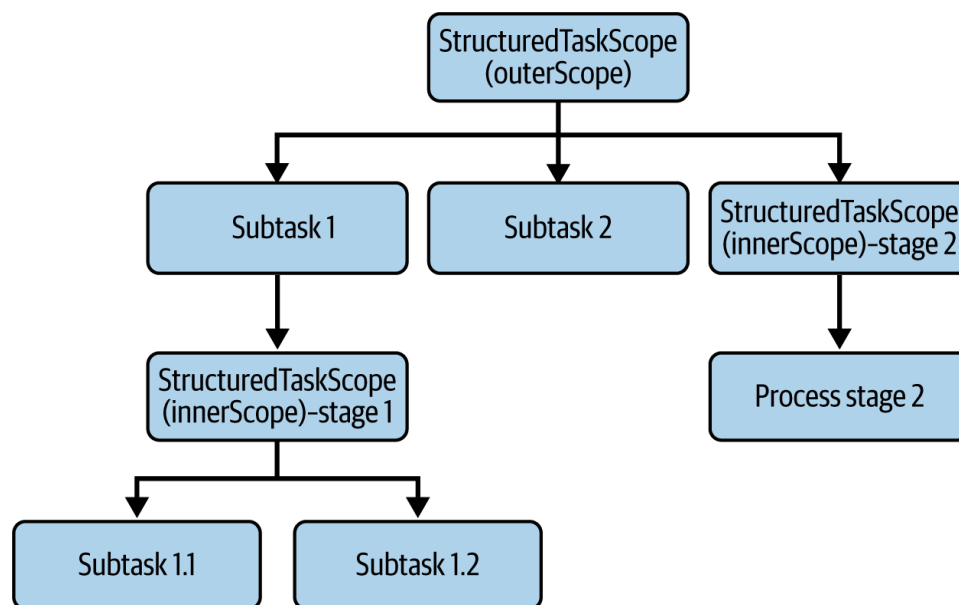


Figure 4-3. The hierarchical structure of tasks in structured concurrency

Let's examine a document processor that gathers content from multiple sources and then performs analysis:

```
public class DocumentProcessor {

    public DocumentReport processDocument(String documentId)
        throws InterruptedException {

        try (var gatheringScope = open(StructuredTaskScope.Joiner.
            <String>allSuccessfulOrThrow())) { ❶
            var headerTask = gatheringScope.fork(() ->
                fetchHeader(documentId)); ❷
            var bodyTask = gatheringScope.fork(() -> fetchBody(documentId));
            var metadataTask = gatheringScope.fork(() ->
                fetchMetadata(documentId)); ❷

            gatheringScope.join(); ❸

            return analyzeContent(headerTask.get(),
                bodyTask.get(),
                metadataTask.get()); ❹
        }
    }
}
```

```

    } catch (StructuredTaskScope.FailedException e) {
        throw new RuntimeException("Failed to gather document content",
    }
}
}

```

In the preceding code, what really happens is:

- ❶ Creates the top-level scope to gather all document components concurrently
- ❷ Launches concurrent tasks to fetch header, body, and metadata from different sources
- ❸ Waits for all content gathering to complete before proceeding to analysis
- ❹ Moves to the analysis phase with all required content available

Now let's examine the nested analysis scope:

```

private DocumentReport analyzeContent(String header,
                                     String body,
                                     String metadata)
    throws InterruptedException {

    try (var analysisScope = open(StructuredTaskScope.Joiner.
        allSuccessfulOrThrow())) { ❶
        var wordCountTask = analysisScope.fork(() ->
            countWords(body)); ❷
        var sentimentTask = analysisScope.fork(() ->
            analyzeSentiment(body)); ❷
        var summaryTask = analysisScope.fork(() ->
            generateSummary(header, body, metadata)); ❷
        analysisScope.join(); ❸
        return new DocumentReport(
            wordCountTask.get(),
            sentimentTask.get(),
            summaryTask.get()
        );
    } catch (StructuredTaskScope.FailedException e) {
        throw new RuntimeException("Failed to analyze document content", e);
    }
}

```

In the preceding code snippets, we have:

- ❶ Creates a second-level scope specifically for performing analysis on the gathered content
- ❷ Launches concurrent analysis tasks for word counting, sentiment analysis, and summary generation

③ Waits for all analysis to complete and constructs the final report

This creates a clean two-level hierarchy:

```
Document Processing (Level 1)
├── Header Fetching (Level 2)
├── Body Fetching (Level 2)
├── Metadata Fetching (Level 2)
└── Content Analysis (Level 2)
    ├── Word Counting (Level 3)
    ├── Sentiment Analysis (Level 3)
    └── Summary Generation (Level 3)
```

Here are the supporting methods that complete our example:

```
private String fetchHeader(String documentId)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(100));
    return "Header for document " + documentId;
}

private String fetchBody(String documentId)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(200));
    return "This is the main content of document " + documentId +
        " with multiple sentences and important information.";
}

private String fetchMetadata(String documentId)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(150));
    return "Created: 2024-01-01, Author: John Doe";
}

private Integer countWords(String content)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(100));
    return content.split("\\s+").length;
}

private String analyzeSentiment(String content)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(200));
    return content.toLowerCase().contains("important") ? "Positive" : "Neutra
}

private String generateSummary(String header, String body, String metadata)
    throws InterruptedException {
    Thread.sleep(Duration.ofMillis(150));
    return header + ": " + body.substring(0, Math.min(50, body.length())) + "
```

```

public record DocumentReport(int wordCount,
                             String sentiment, String summary) {

    @Override
    public String toString() {
        return String.format("Document Report:\n Words: %d\n " +
                             "Sentiment: %s\n Summary: %s",
                             wordCount, sentiment, summary);
    }
}

```

Now let's use the following main method and run the above code:

```

void main() {
    var processor = new DocumentProcessor();
    try {
        var report = processor.processDocument("DOC-123");
        System.out.println(report);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.out.println("Processing interrupted");
    }
}

```

When we run this, we will see output like:

```

Document Report:
Words: 14
Sentiment: Positive
Summary: Header for document DOC-123: This is the main content of document
123 with ...

```

This nested approach demonstrates structured concurrency's elegance. Each scope manages its own lifecycle and resources, creating a clean hierarchy where failures propagate naturally, resources are automatically cleaned up, and cancellation cascades properly from parent to child scopes.

Observability

As we have learned, structured concurrency provides an inherent structure that imposes a clear hierarchical relationship between tasks and their subtasks. This hierarchy mirrors the nested structure of the code, making it much easier to understand and reason about the flow of concurrent execution.

This treelike structure becomes extremely useful for debugging. Traditional thread dumps often present a chaotic jumble of threads with

no clear relationships. In contrast, structured thread dumps provide a hierarchical view of our concurrent system, making it easier to identify issues.

We can generate structured thread dumps in JSON format using the `jcmd` tool:

```
jcmd <pid> Thread.dump_to_file -format=json <output_file>
```

These dumps reveal valuable information, including task hierarchy and subtask states.

Let's look at an example using our document processor:

```
import module java.base;
import static java.util.concurrent.StructuredTaskScope.open;

public class DocumentProcessor {
    private final ThreadFactory threadFactory = Thread.ofVirtual()
        .name("doc-proc", 1)
        .factory();

    public DocumentReport processDocument(String documentId)
        throws InterruptedException {
        try (var gatheringScope = open(
            StructuredTaskScope.Joiner.<String>allSuccessfulOrThrow(),
            conf -> conf.withThreadFactory(threadFactory)
                .withName("doc-gathering-scope"))) { ❶

            var headerTask = gatheringScope.fork(() ->
                fetchHeader(documentId)); ❷
            var bodyTask = gatheringScope.fork(() ->
                fetchBody(documentId)); ❷
            var metadataTask = gatheringScope.fork(() ->
                fetchMetadata(documentId)); ❷

            gatheringScope.join(); ❸

            return analyzeContent(headerTask.get(),
                bodyTask.get(), metadataTask.get());
        } catch (StructuredTaskScope.FailedException e) {
            throw new RuntimeException("Failed to gather document content",
                e);
        }
    }

    private String fetchHeader(String documentId) throws InterruptedException {
        Thread.sleep(Duration.ofSeconds(10)); ❹
        return "Header for document " + documentId;
    }
}
```

```

        private String fetchBody(String documentId) throws InterruptedException {
            Thread.sleep(Duration.ofSeconds(10));
            return "This is the main content of document " + documentId;
        }

        private String fetchMetadata(String documentId) throws InterruptedException {
            Thread.sleep(Duration.ofSeconds(10));
            return "Created: 2024-01-01, Author: John Doe";
        }
    }
}

```

We have slightly changed our program to demonstrate the thread dump:

- ❶ We give the scope a descriptive name and use a custom `ThreadFactory` with named threads to make identification easier in thread dumps.
- ❷ Launches parallel tasks for fetching different parts of the document.
- ❸ Waits for all gathering operations to complete.
- ❹ We simulate slow operations that would show up clearly in a thread dump.

If we run this code and generate a thread dump while the operations are in progress, we get a beautifully structured view:

```

{
  "threadDump": {
    "processId": "10259",
    "time": "2025-06-10T00:49:22.146091Z",
    "runtimeVersion": "25-ea+25-3096",
    "threadContainers": [
      {
        "container": "<root>",
        "parent": null,
        "owner": null,
        "threads": [
          {
            "tid": "3",
            "name": "main",
            "stack": [
              "java.base/jdk.internal.misc.Unsafe.park(Native Method)",
              "java.base/java.util.concurrent.locks.LockSupport.park(LockSupport.java:369)",
              "java.base/jdk.internal.misc.ThreadFlock.awaitAll(ThreadFlock.java:305)",
              "java.base/java.util.concurrent.StructuredTaskScopeImpl.join(StructuredTaskScopeImpl.java:243)",
              "ca.bazlur.mcj.chap4.DocumentProcessor.processDocument(DocumentProcessor.java:23)"
            ]
          }
        ]
      }
    ]
  }
}

```

```

    }
  ],
  "threadCount": "8"
},
{
  "container": "doc-gathering-scope/jdk.internal.misc.ThreadFlock$Thr
tainerImpl@78fd3572",
  "parent": "<root>",
  "owner": "3",
  "threads": [
    {
      "tid": "36",
      "name": "doc-proc2",
      "stack": [
        "java.base/java.lang.Thread.sleep(Thread.java:574)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.fetchBody(DocumentProc
java:69)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument
umentProcessor.java:20)"
      ]
    },
    {
      "tid": "38",
      "name": "doc-proc3",
      "stack": [
        "java.base/java.lang.Thread.sleep(Thread.java:574)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.fetchMetadata(Document
sor.java:76)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument
umentProcessor.java:21)"
      ]
    },
    {
      "tid": "26",
      "name": "doc-proc1",
      "stack": [
        "java.base/java.lang.Thread.sleep(Thread.java:574)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.fetchHeader(DocumentPr
r.java:63)",
        "ca.bazlur.mcj.chap4.DocumentProcessor.lambda$processDocument
umentProcessor.java:19)"
      ]
    }
  ],
  "threadCount": "3"
}
]
}
}

```

The thread dump reveals several key insights:

- We can clearly see our named scope `doc-gathering-scope` as a separate container with its parent being `<root>` and owner being thread 3 (the main thread).
- All three subtasks (`doc-proc1`, `doc-proc2`, `doc-proc3`) are clearly grouped under our document-gathering scope, making it easy to understand which threads belong to which logical operation.
- Each thread's stack trace shows exactly what operation it's performing — `fetchHeader`, `fetchBody`, and `fetchMetadata` —making debugging straightforward.
- The `owner: 3` field shows that the main thread (`tid 3`) owns this scope, establishing the clear parent-child relationship.

Tools can now analyze these structured thread dumps to reveal even more insights. For example, the simple tool [Threadly](#) can parse the JSON thread dump and visualize the tree structure of structured concurrency, making it even easier to understand the hierarchical relationships and debug complex concurrent applications.

While the `jcmd` tool offers a convenient way to generate structured thread dumps, there might be situations where we need more control or want to automate the process. The `HotSpotDiagnosticMXBean` comes in handy here. We can use it to trigger thread dumps programmatically whenever a specific condition arises, such as an exception within a `StructuredTaskScope`.

Here's an example of how we might use it:

```
public void processWithErrorHandling(String documentId) {
    try (var scope = open(StructuredTaskScope.Joiner.
        <String>allSuccessfulOrThrow(),
        cf -> cf.withName("error-prone-scope"))) {
        scope.fork(() -> {
            if (new Random().nextBoolean()) { ❶
                throw new RuntimeException("Simulated failure");
            }
            return fetchHeader(documentId);
        });
        scope.join();
    } catch (StructuredTaskScope.FailedException e) {
        HotSpotDiagnosticMXBean bean = ManagementFactory
            .getPlatformMXBean(HotSpotDiagnosticMXBean.class); ❷
        try {
            Path path = Path.of("./structured-concurrency-error.json");
            bean.dumpThreads(path.toAbsolutePath().toString(),
                HotSpotDiagnosticMXBean.ThreadDumpFormat.JSON); ❸
            System.out.println("Thread dump captured: " + path);
        } catch (IOException ex) {
            throw new RuntimeException("Failed to generate thread dump", ex);
        }
        throw new RuntimeException("Processing failed", e);
    }
}
```

```

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new RuntimeException("Processing interrupted", e);
    }
}

```

We have done a few things in this code:

- ❶ We randomly introduce failures to trigger the error handling and thread dump generation.
- ❷ We obtain a reference to the `HotSpotDiagnosticMXBean` for programmatic thread dump generation.
- ❸ When an error occurs, we automatically generate a JSON thread dump for later analysis.

So we can conclude that structured concurrency not only simplifies concurrent programming but also enhances debugging and maintenance, providing a clearer view of task execution and relationships.

In Closing

Structured concurrency represents a significant advancement in concurrent programming for Java. By embracing the principle of tying subtask lifecycles to their parent tasks, we've unlocked a new level of reliability, maintainability, and observability in our concurrent applications.

Throughout this chapter, we've seen how structured concurrency addresses fundamental challenges in concurrent programming. Clear resource management ensures automatic cleanup of subtasks, preventing the resource leaks and zombie threads that plagued traditional concurrent code. Hierarchical error handling allows failures to propagate naturally through the task hierarchy, making error handling predictable and comprehensive. Enhanced debugging capabilities provide structured thread dumps with clear visibility into task relationships, transforming debugging from guesswork into systematic analysis. Custom joiners enable sophisticated coordination strategies while maintaining the safety guarantees of structured concurrency. Perhaps most importantly, the nested structure mirrors business logic, making concurrent code as readable and maintainable as sequential code.

A common question that arises is whether structured concurrency can be used with both virtual and platform threads. The answer is yes, structured concurrency is compatible with both, offering flexibility to developers. We can easily configure `StructuredTaskScope` with platform threads by providing a custom `ThreadFactory`:

```

public void demonstratePlatformThreads() throws InterruptedException {
    ThreadFactory platformFactory = Thread.ofPlatform()
        .name("platform-worker-", 0)
        .factory();

    try (var scope = open(Joiner.allSuccessfulOrThrow(),
        cf -> cf.withThreadFactory(platformFactory))) {

        scope.fork(() -> performTask("Task 1"));
        scope.fork(() -> performTask("Task 2"));

        scope.join();
    }
}

```

However, this brings us back to the original problem: the scarcity of platform threads. We must be mindful of the potential scalability and resource constraints associated with platform threads in high-concurrency scenarios. Virtual threads remain the preferred choice for most structured concurrency applications, offering the scalability benefits that make this programming model truly powerful.

Whether you're managing incoming server requests, coordinating microservice calls, or dealing with complex multistage workflows, structured concurrency provides a powerful and intuitive framework for navigating the complexities of concurrent programming. By adopting this paradigm, you can write code that is not only more efficient and robust but also easier to understand and ultimately more enjoyable to work with.